

Un Supervised Learning

Unsupervised Machine Learning

Unsupervised Machine Learning is a type of machine learning where the algorithm is **trained** on **unlabelled data**, meaning the data does not have **predefined categories or outcomes**.

The goal is to identify **hidden patterns, relationships, or structures** within the dataset.

.....

Techniques:

- **Clustering:** Grouping data into clusters based on similarity (**e.g., K-Means**)
- **Dimensionality Reduction:** Reducing the number of **features** in a dataset while retaining essential **features** (**e.g., PCA**)
- **Association Rule Learning:** Discovering **relationships** between **variables** (**e.g., Apriori**)

Real time-Use case

Customer Segmentation in Retail industries:

- Data like **transaction history**, **frequency**, **product preferences**, and **spending** is collected.
- Clustering algorithms (e.g., **K-Means**) identify **distinct customer groups**.
- **Personalized marketing campaigns** for each group.

Clustering Methods

1. Partition Methods

K-Means

K-Medoids

2. Hierarchical Methods

Hierarchical Clustering

CURE (Clustering Using Representatives)

Chameleon

3. Density-Based Methods

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

OPTICS

.....

4. Grid-Based Methods

STING (Statistical Information Grid)

CLIQUE

5. Model-Based Methods

Gaussian Mixture Models (GMM)

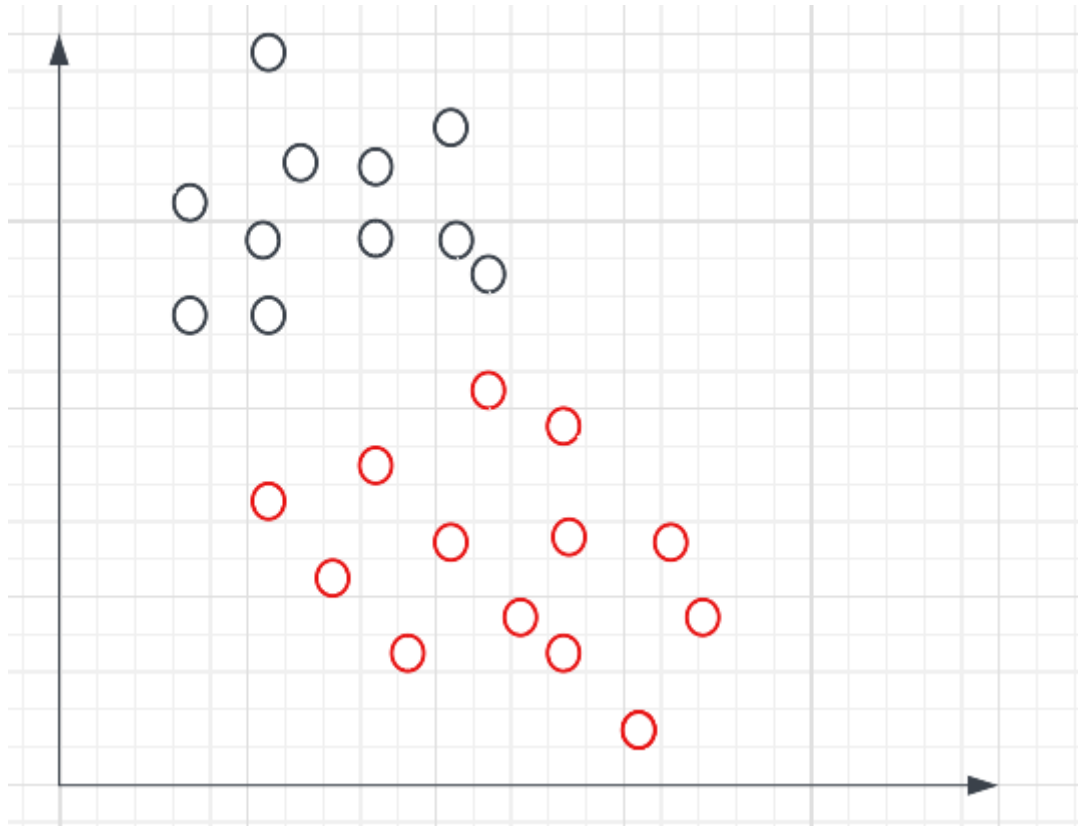
Expectation-Maximization (EM)

K-Means Clustering

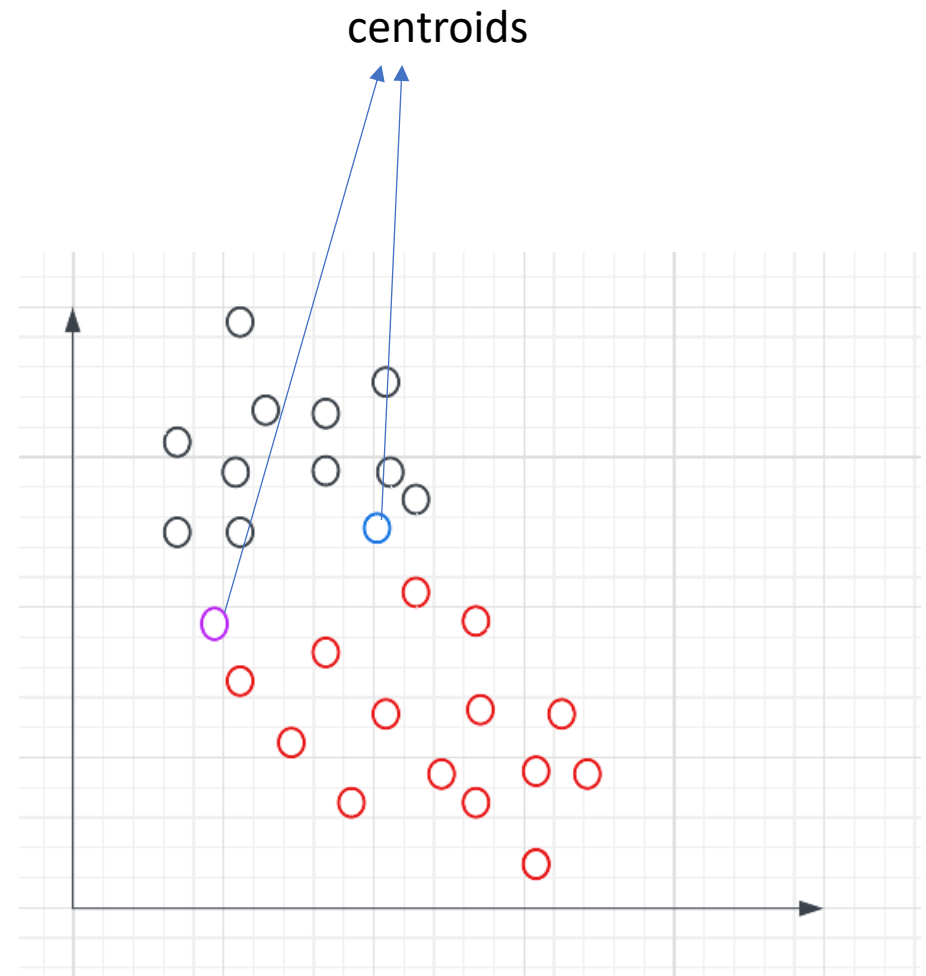
- **K-Means Clustering** is an **unsupervised machine learning algorithm** that partitions **data** into **K clusters**, where each data point belongs to the cluster with the nearest mean (**centroid**).
- The algorithm **iteratively** refines **cluster centroids** to minimize the variance (**Smaller variance indicates tighter clusters**) within clusters.
- So that the **members of a cluster** are **similar** to each other and **distinct** from other **clusters**.

K-Means algorithm steps

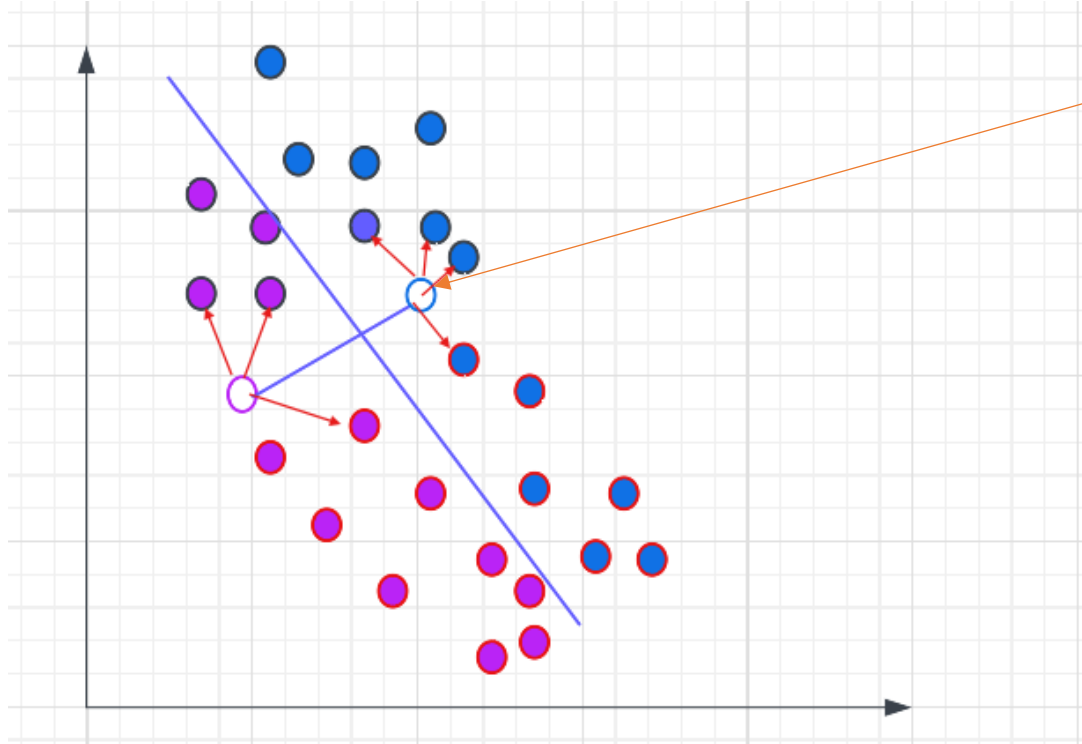
Data points



1. Initialize some **K** values, i.e.
called as centroid here.
Say **k** value = 2



.....



Using **Euclidian Distance** or **Manhattan distance** calculate the distance between **centroid** and data points .

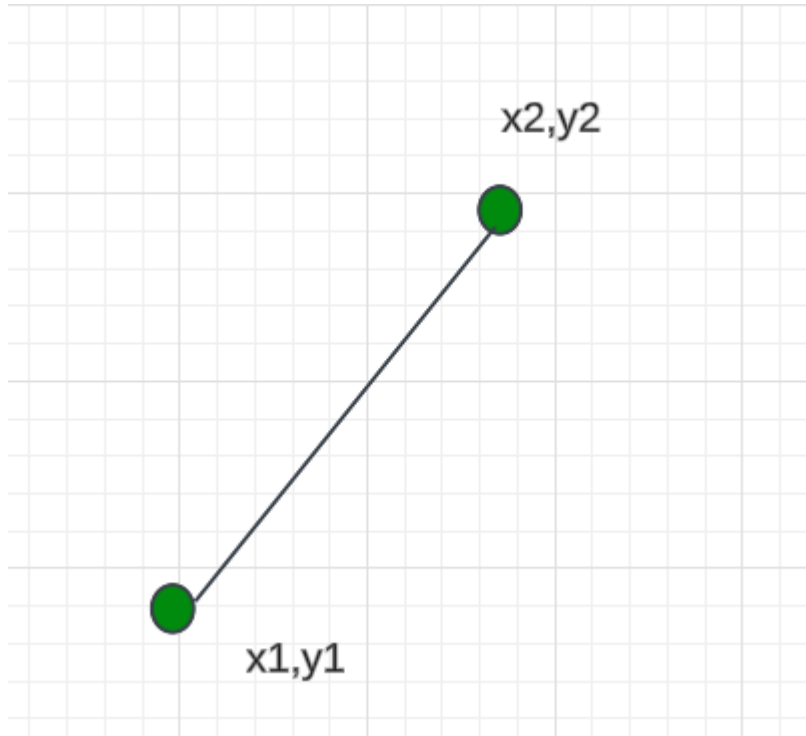
Points which are **closer** to the centroid are belongs to one particular **group**.

Draw a **perpendicular line** between **two distance**. The points **below the line** belongs to **purple colour centroid** , the points **above line** belongs to **blue colour centroid**.

.....

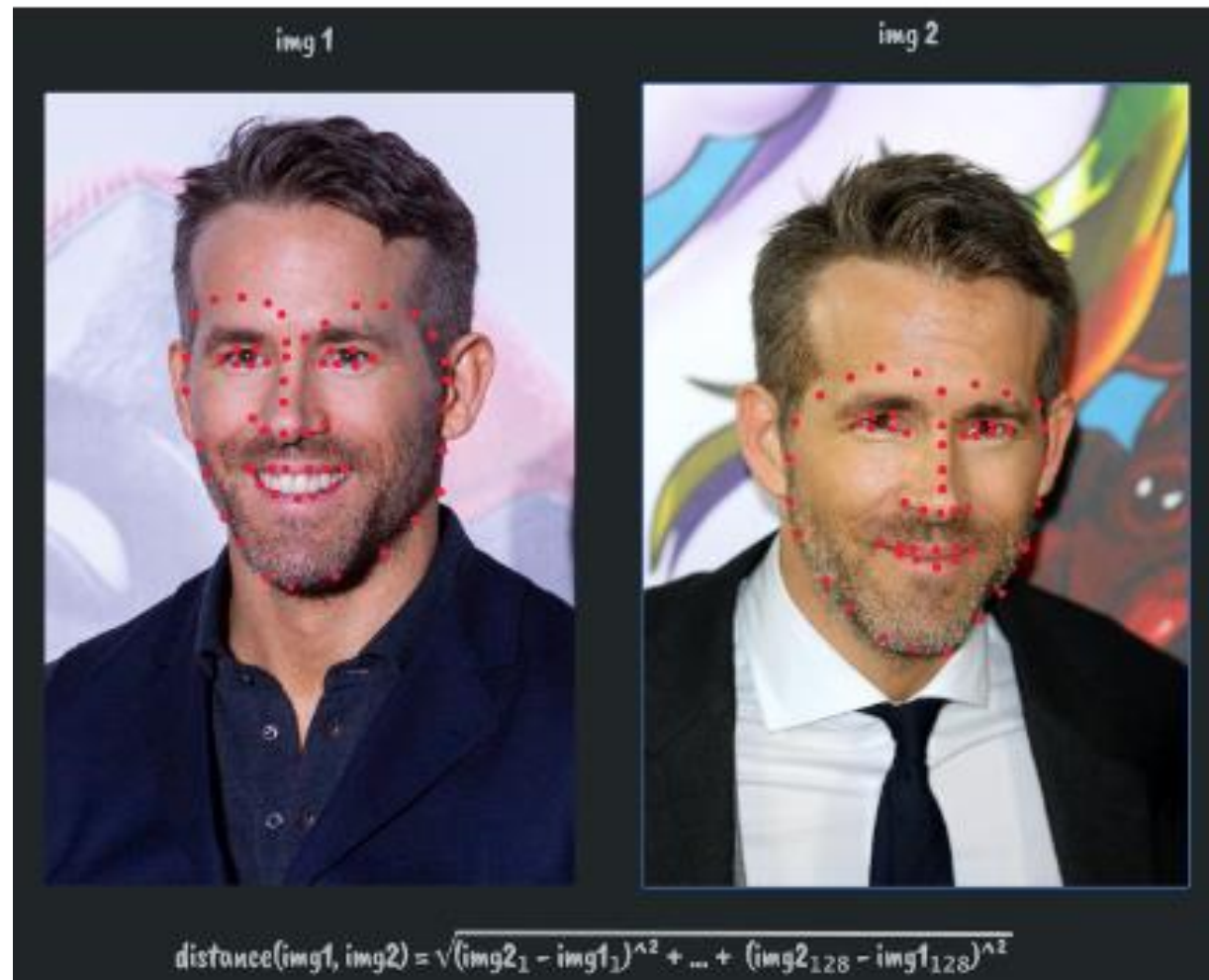
To calculate the K-Nearest neighbour we will use **distance formula**.

1.Euclidian distance



$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Facial recognition



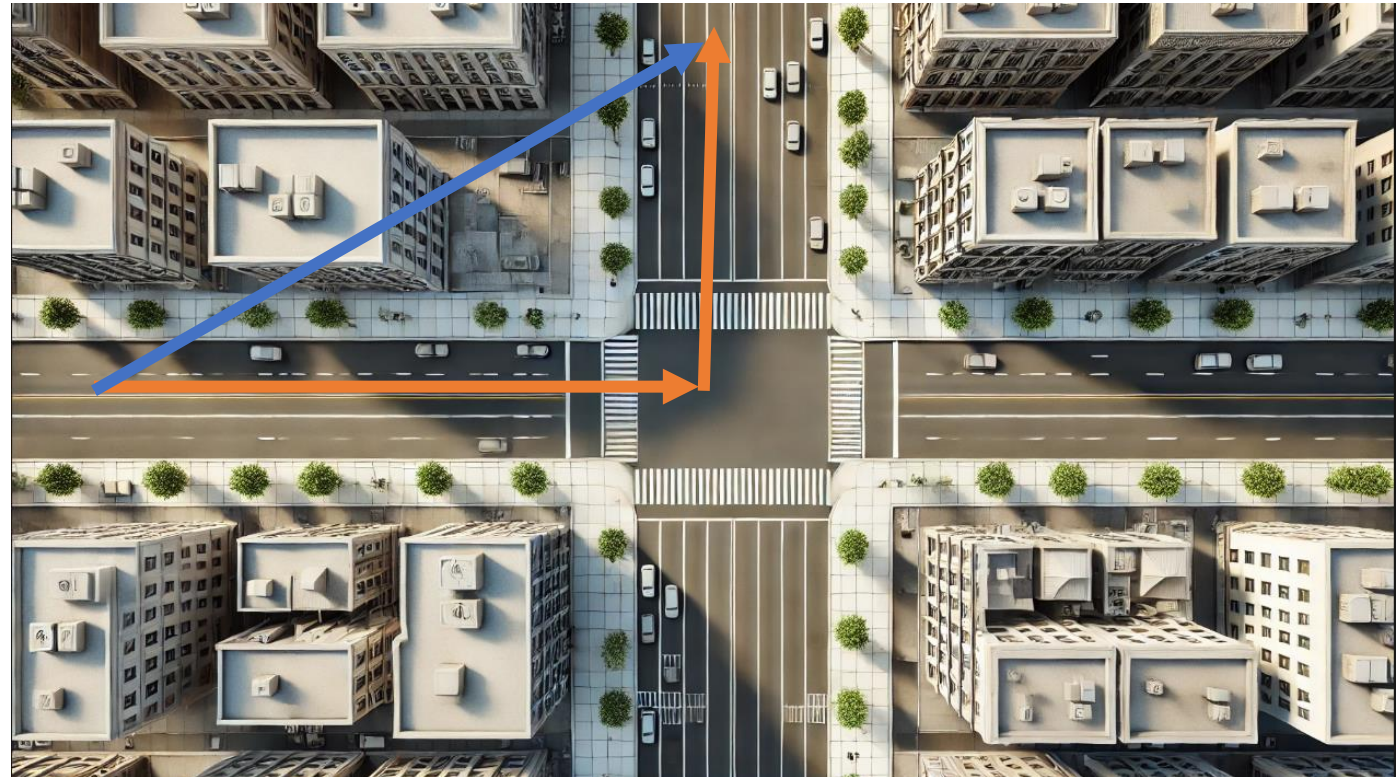
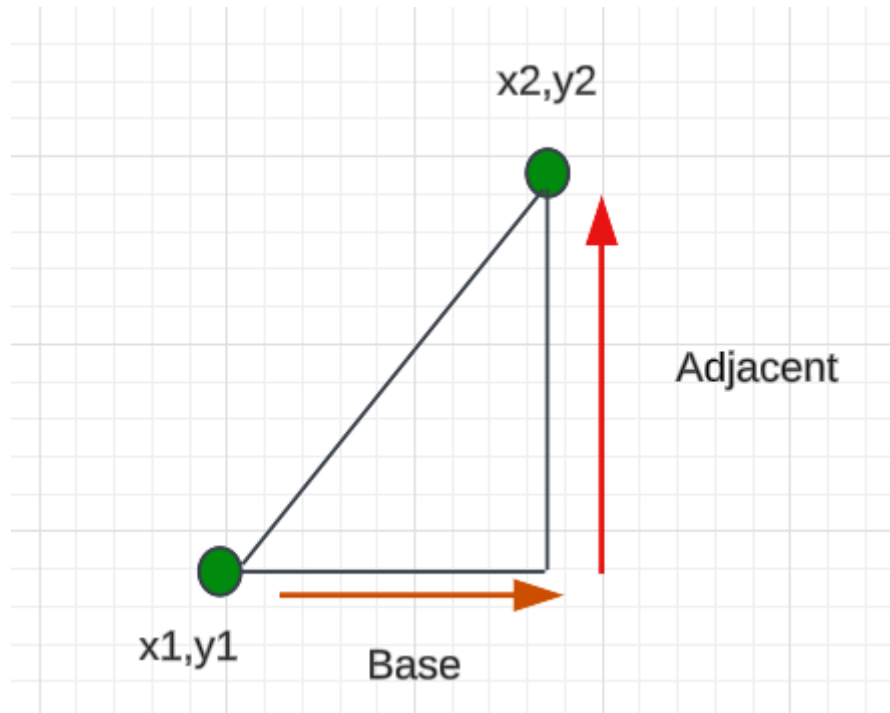
Two images are compared by calculating the **Euclidean distance** between their **feature vectors**.

.....

2. Manhattan Distance

Also known "taxicab" distance

$$\text{Distance} = |x_2 - x_1| + |y_2 - y_1|$$

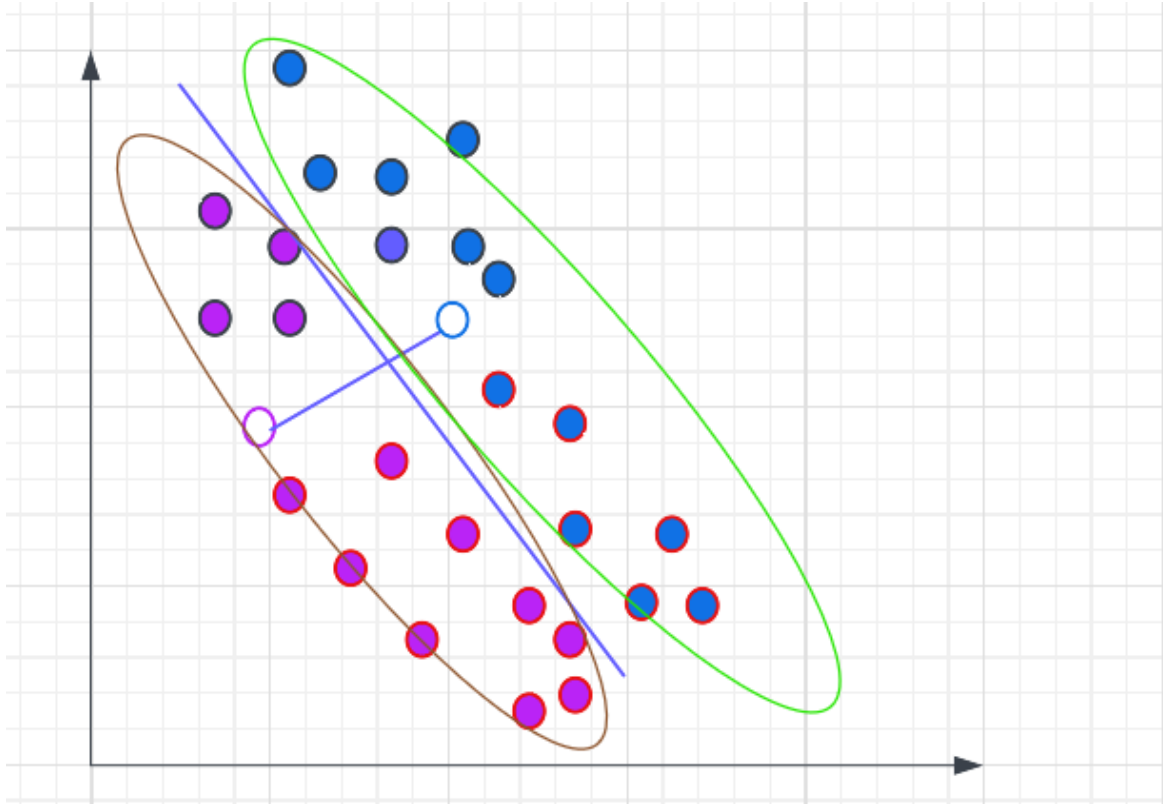


City Block Distance



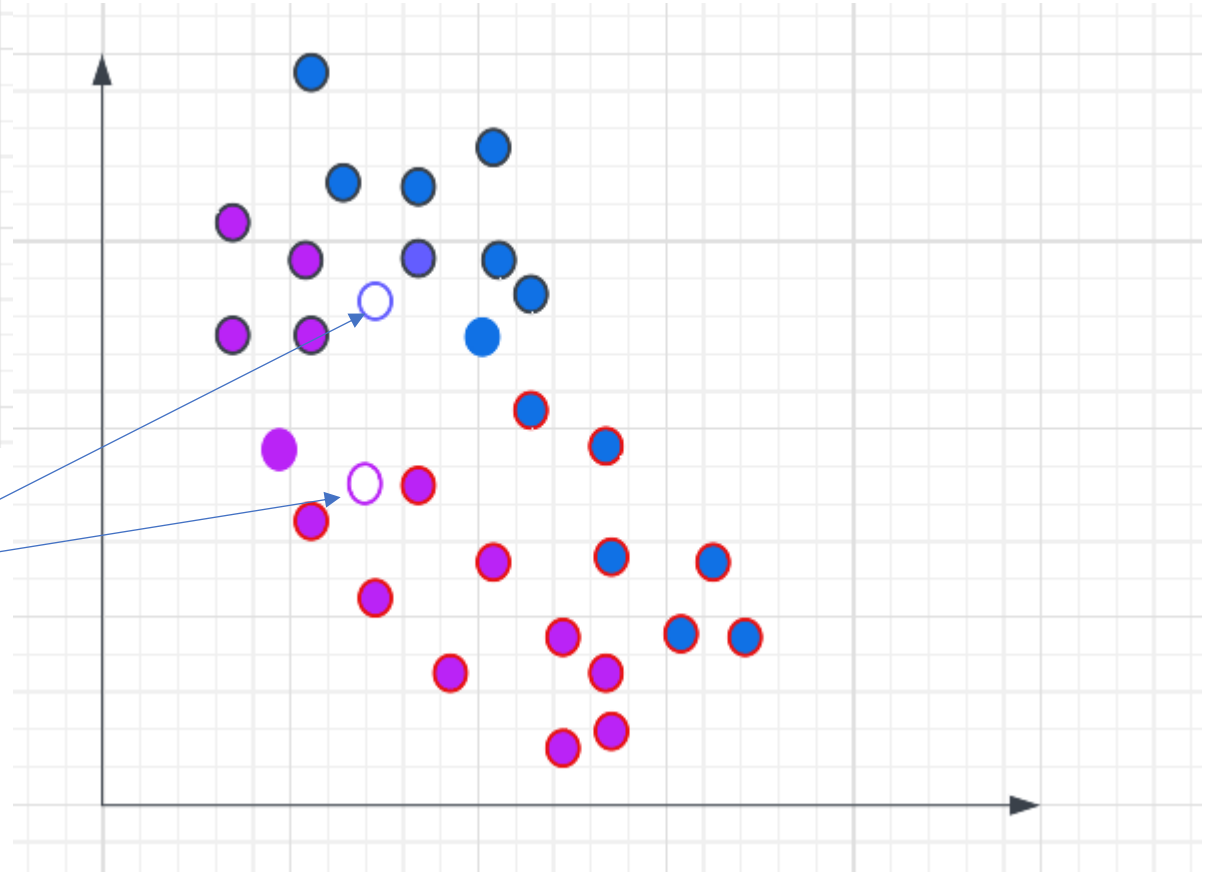
Manhattan distance is ideal since the vehicles follow a **grid pattern**, **and diagonal shortcuts are not allowed**.

.....

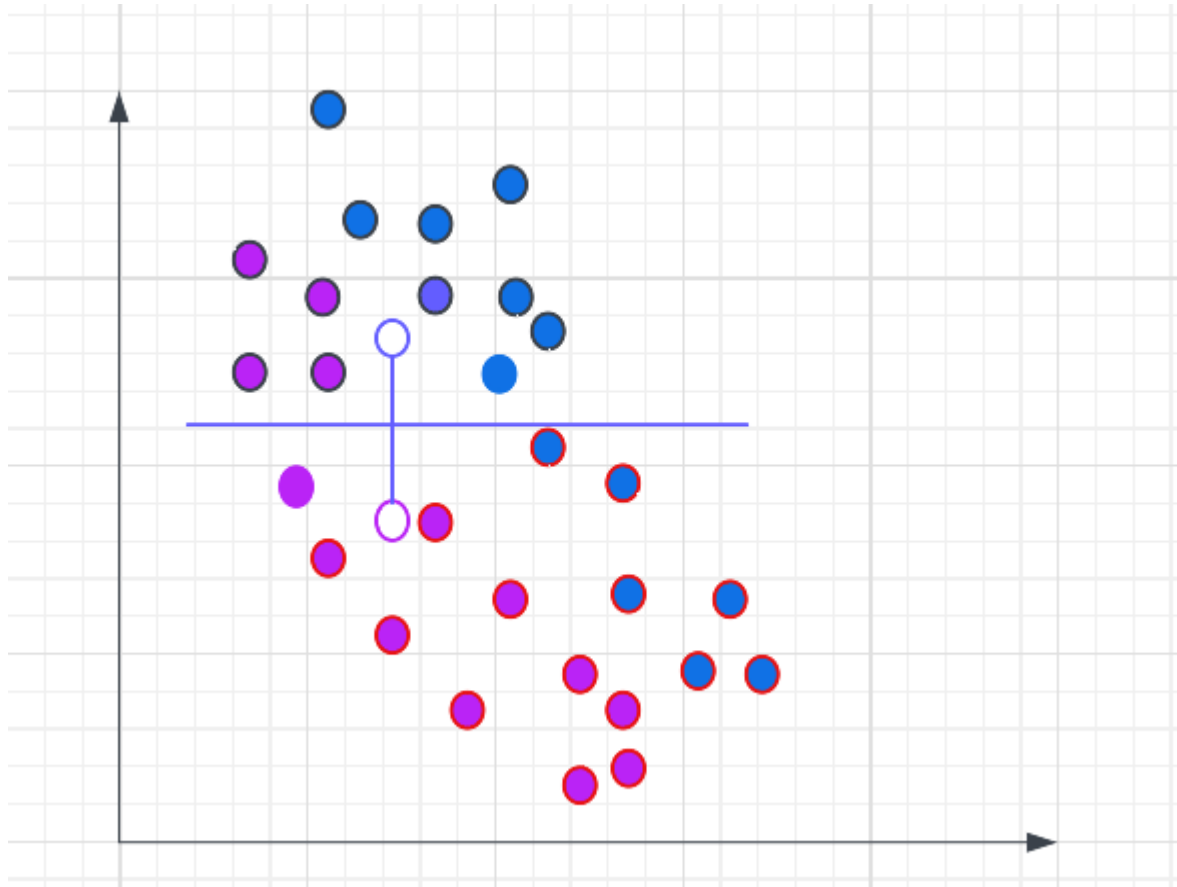


Take the average of this two clusters separately.
The average value is the **new centroid**

New centroids



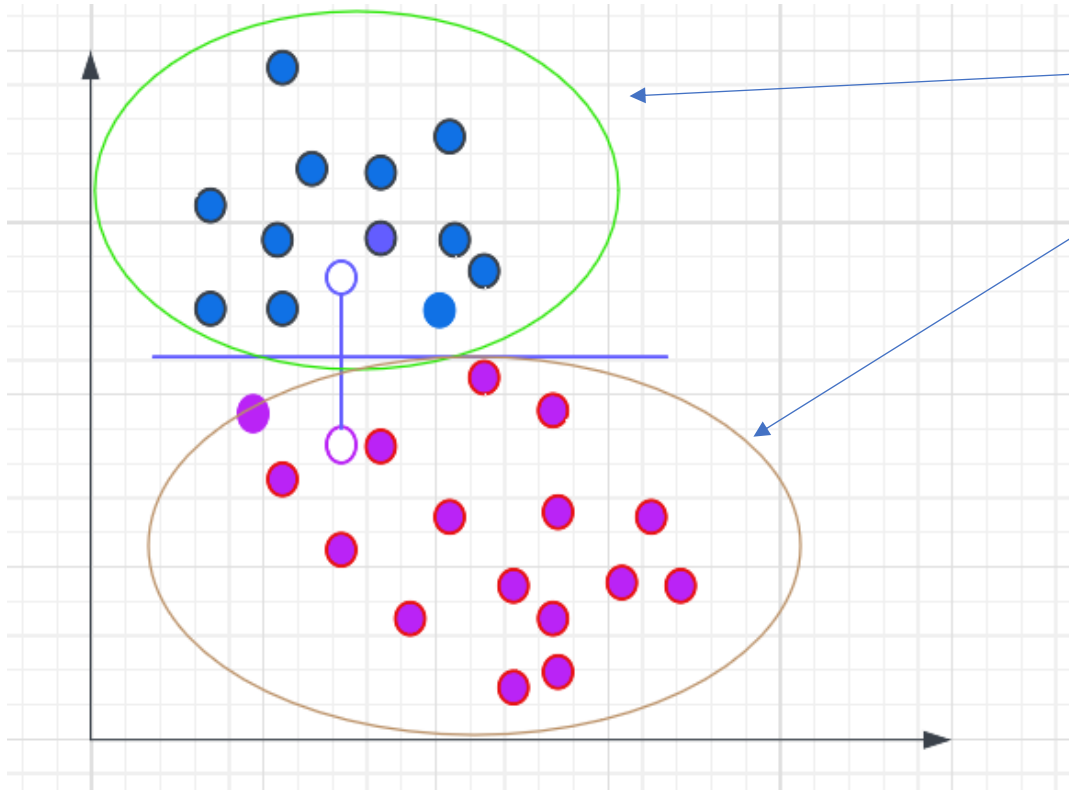
.....



The points above the perpendicular line will belong to **new blue colour centroid**.

Points below that will belong to **new purple colour centroid**

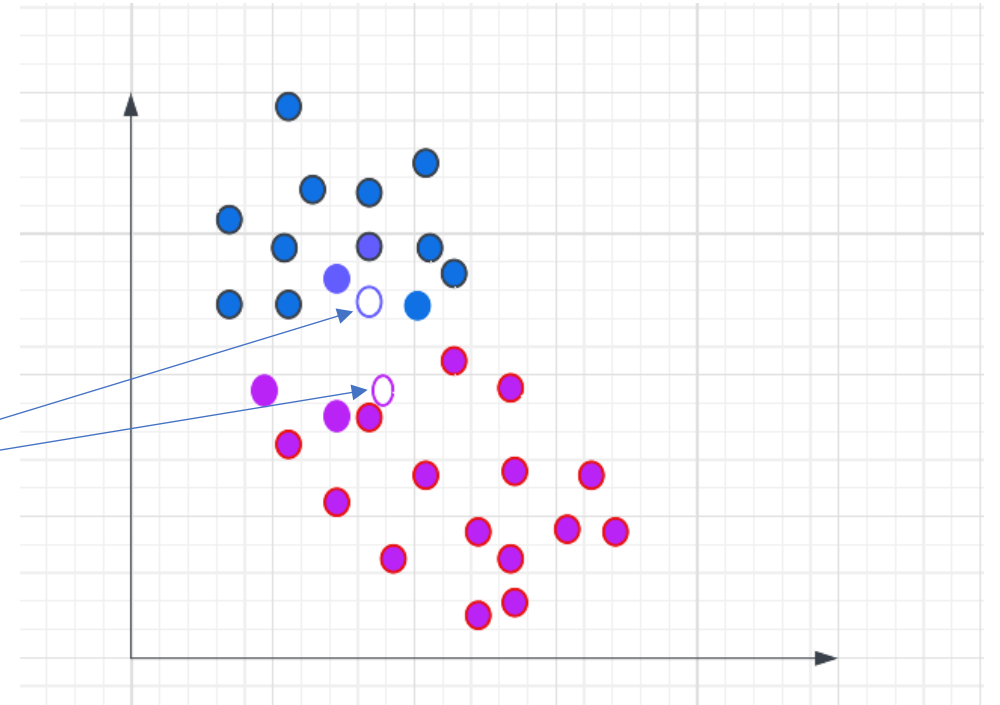
.....



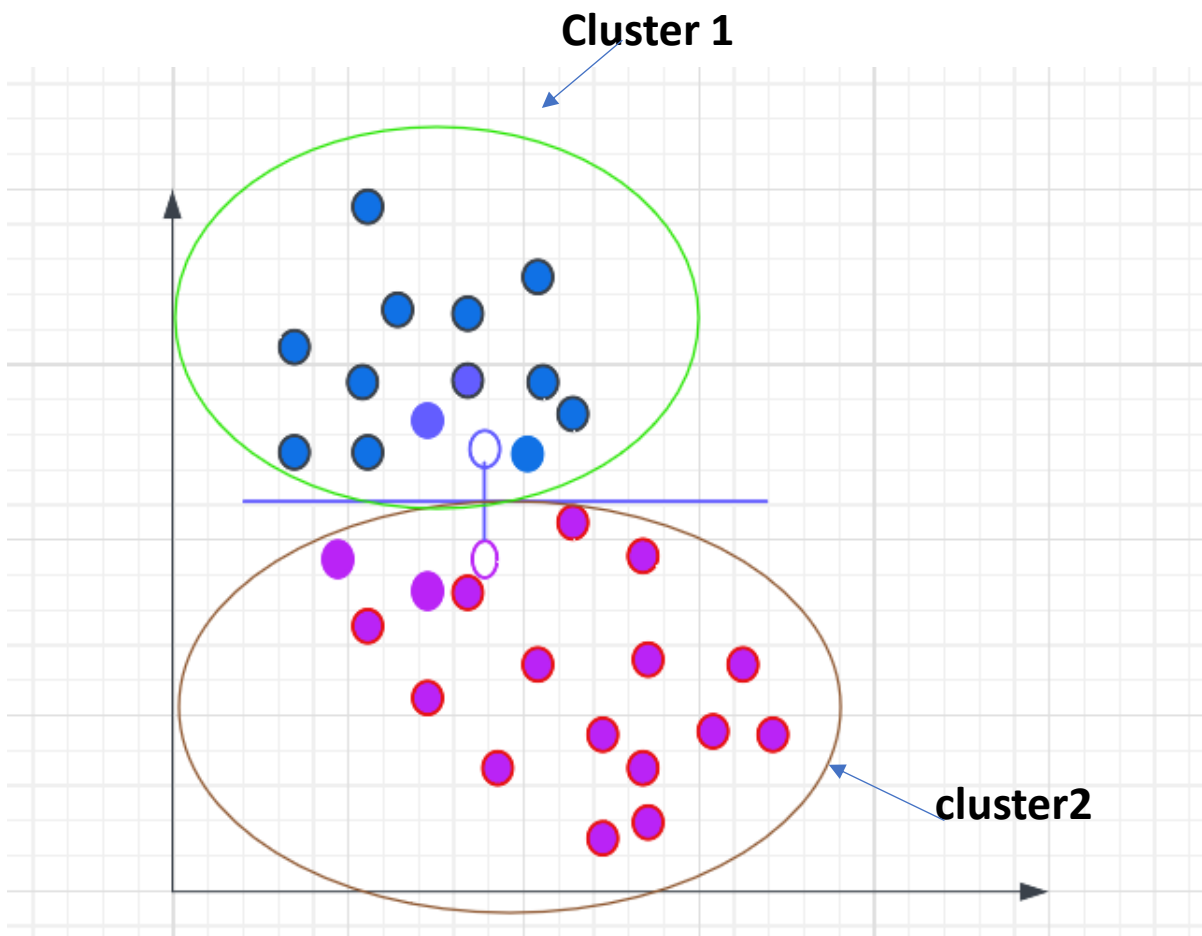
Again **average** has to be computed between the two groups.

And **two new centroid** is assigned based on the **mean** value.

New centroids



.....



The **points** didn't change , and it clearly forms two different clusters .

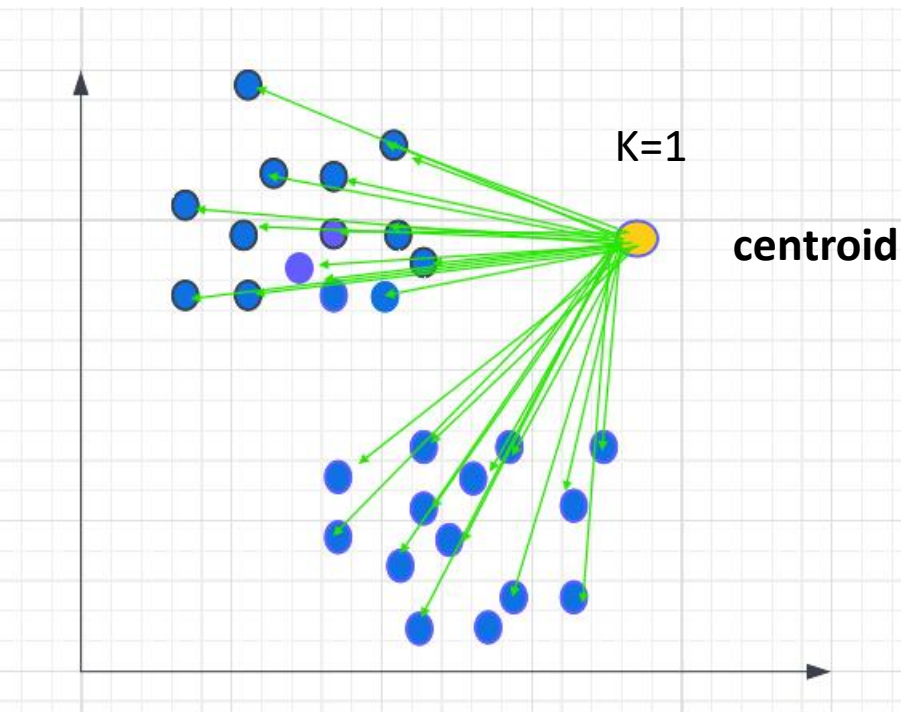
Since we get two different clusters, We are going to **stop** the process here.

How do we select the K values?

Wcss --- Within cluster sum of squares

It quantifies how **tightly** the **data points in each cluster** are grouped around their **centroid**.
The goal of K-Means is to **minimize WCSS**, ensuring compact and well-separated clusters.

Initialize k = 1 to 20



Calculate the distance between **centroid** and **all the data points** using **Wcss**

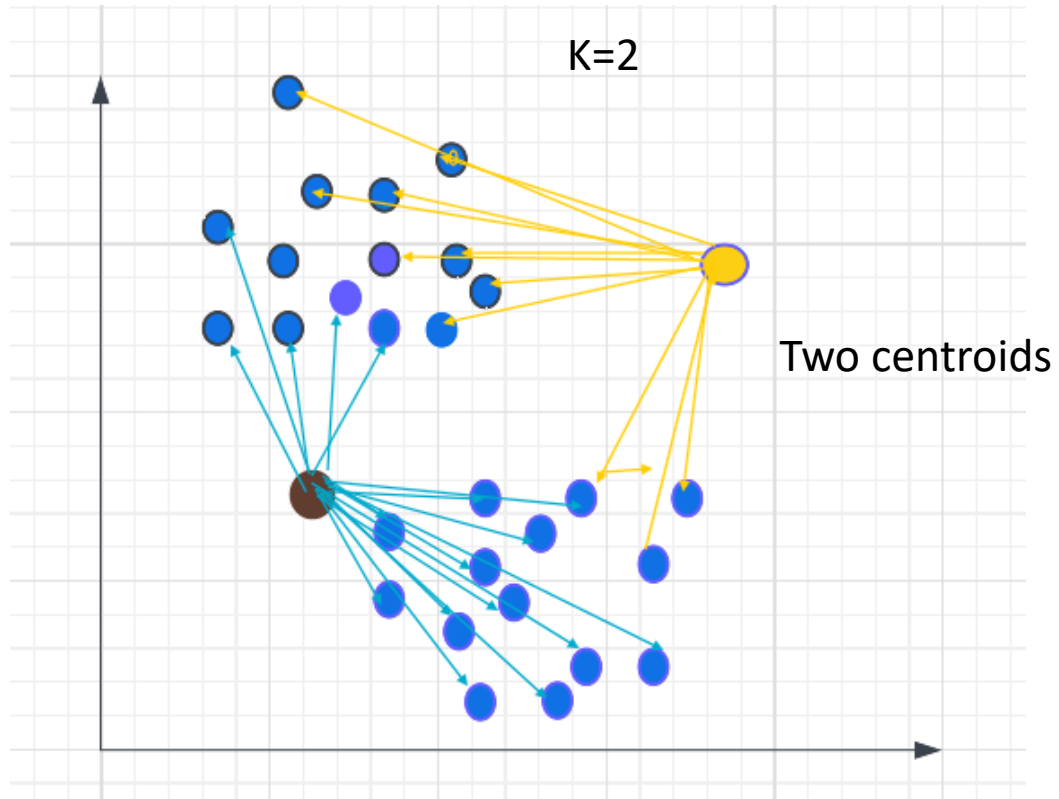
$$wcsc = \sum_{k=1}^k (\text{distance between points to nearest centroids})^2$$

$$= \sum_{k=1}^k (x - \mu)^2$$

Data point

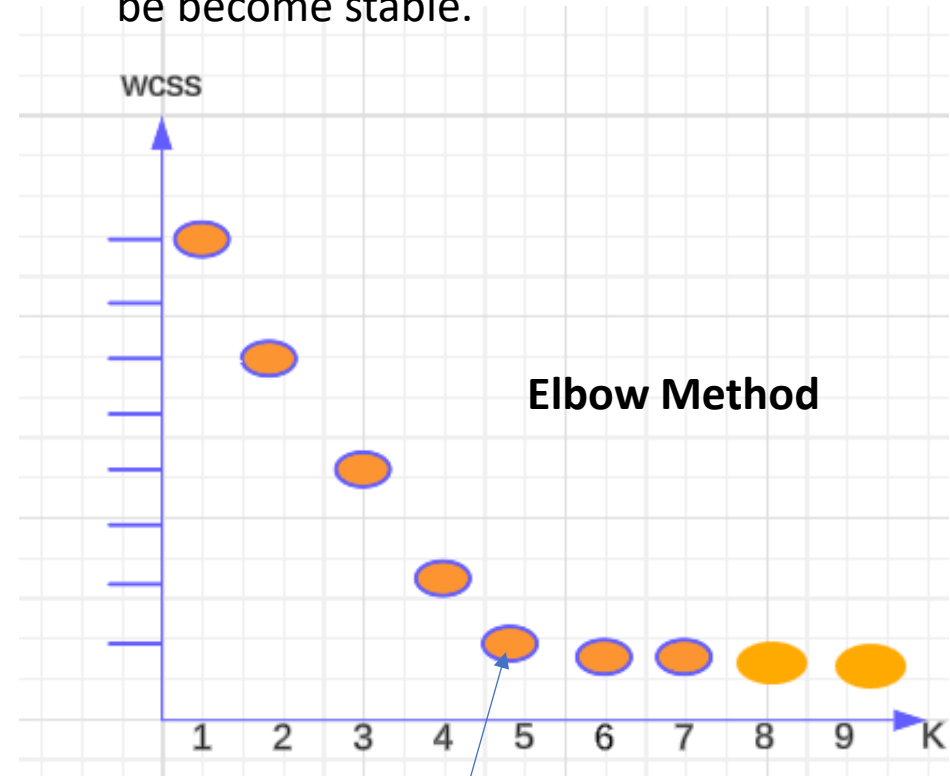
Centroid of cluster

.....



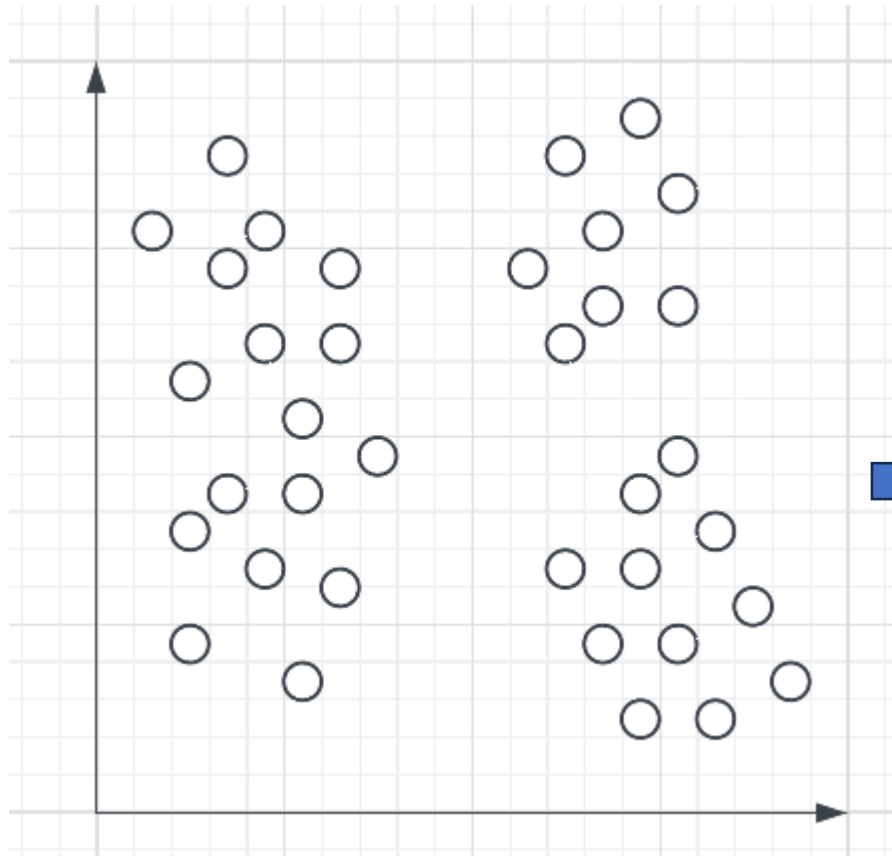
Calculate the distance between two **centroid** and all the **data points** using **Wcss**

When the **K value increases** the **Wcss value will decrease**. At one point it will be become stable.

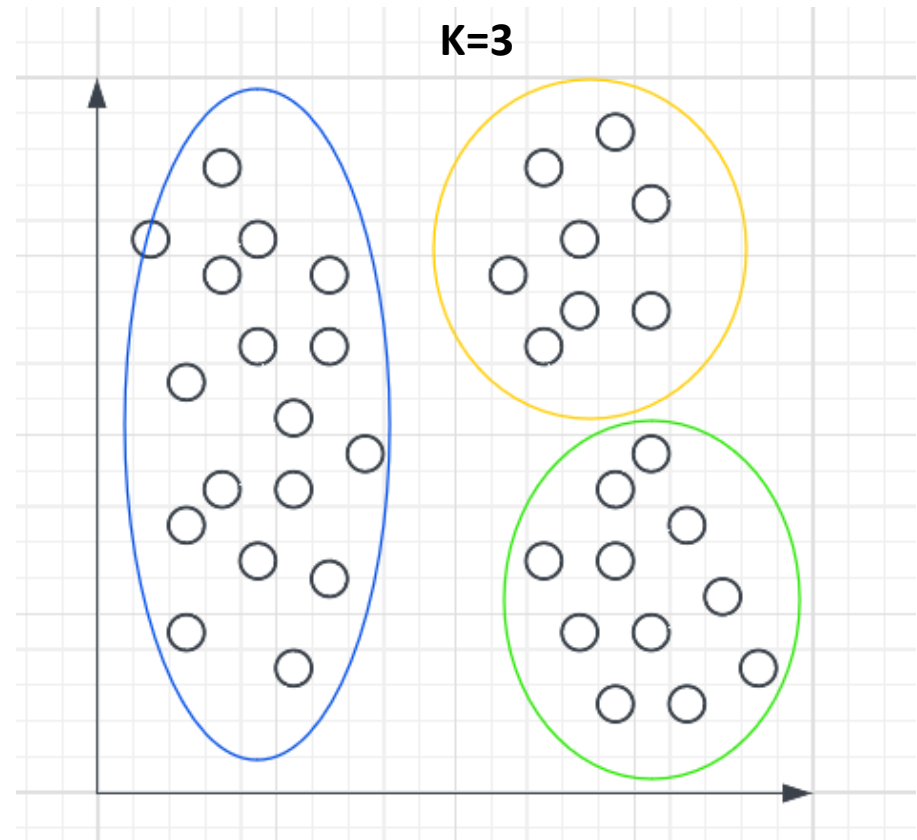


This is the **knee** point from where the **wcss** value become stable.
So **5** is the **efficient K value**.

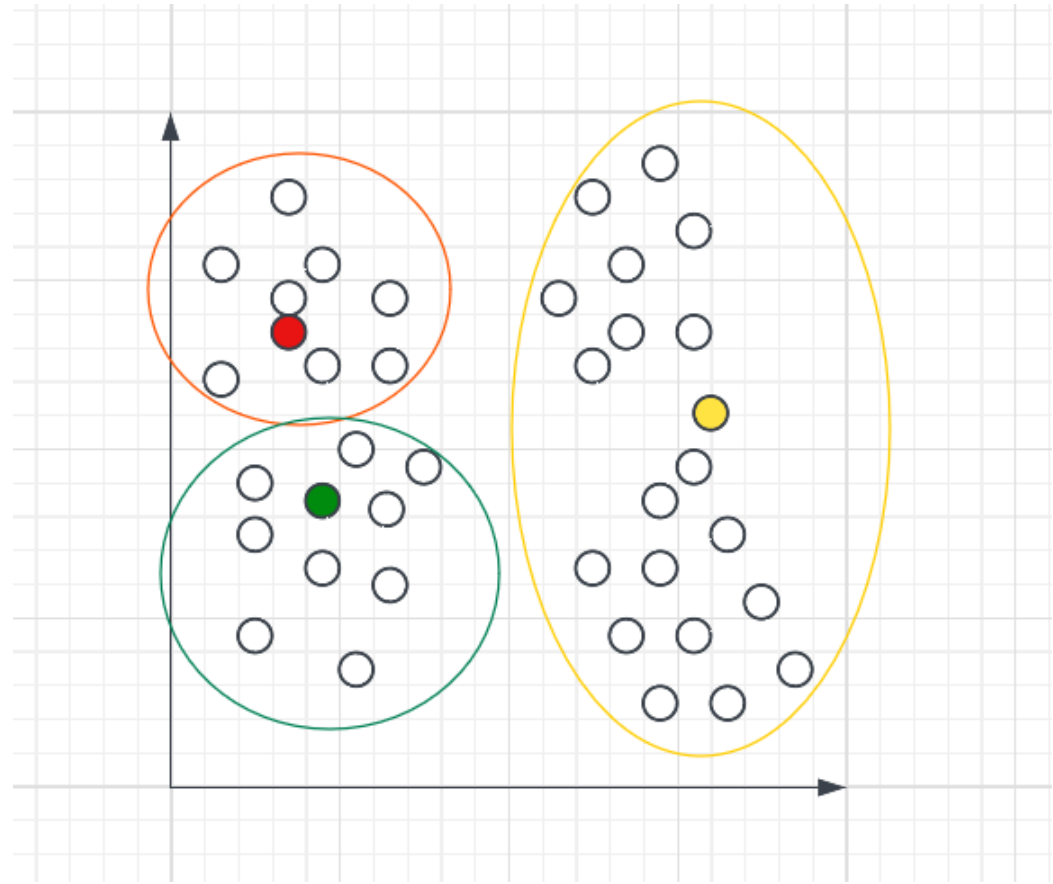
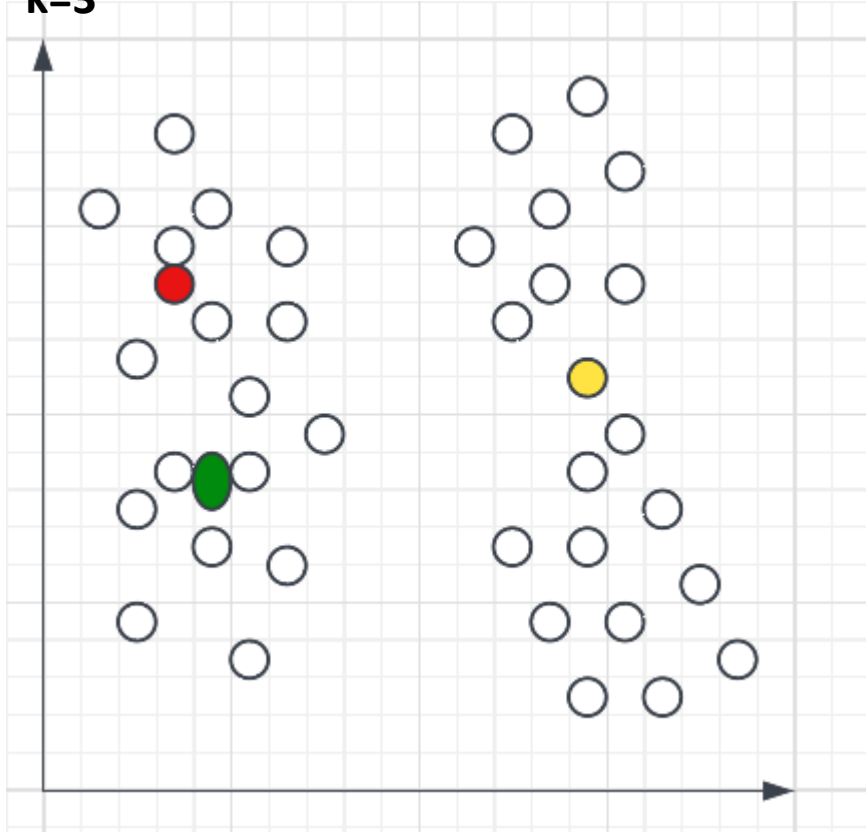
Random Initialization Trap



K -Means

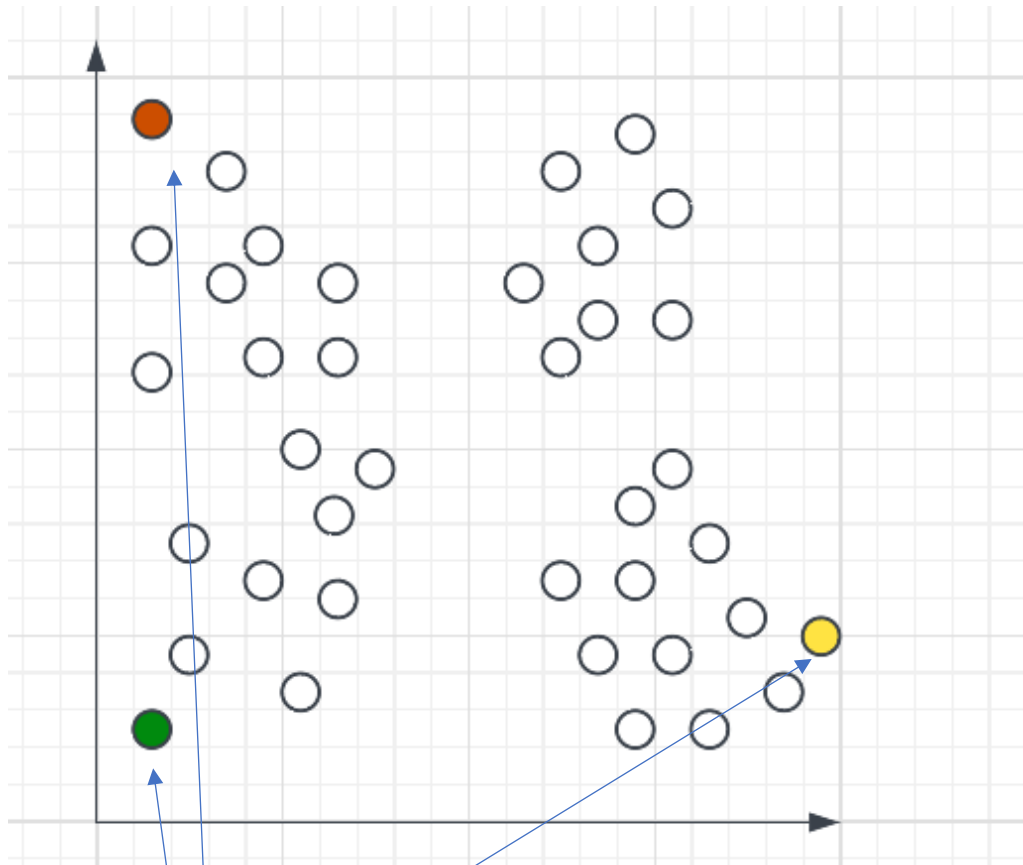


Random initialization of **3 centroids**,
k=3



When ever the **centroid is initialized randomly**, we may get **different group of clusters**.

To overcome this we can use **k-means ++**



In the sample space

Centroids will be initialized far away from each other.

Instead of random initialization, **KMeans++** carefully chooses initial centroids to avoid poor clustering due to **bad starting points**.

Improves Clustering Accuracy:

By reducing the risk of **bad initialization**, **KMeans++** often produces **better final clusters** compared to the standard K-Means.

.....

K-Means Clustering Steps:

1. Choose the number of clusters (k):

Select the **number of clusters(K)** in which you want to group the data.

2. Initialize the centroids:

Randomly select **k data points** from the dataset as the **initial cluster centroids**.

These centroids are the **starting points** that represent the **center** of each cluster.

3. Assigning data points to the nearest centroid:

For each data point in the dataset, **calculate the distance (Euclidean distance)** between the point and **each centroid**.

Assign the **point** to the cluster whose **centroid is closest**.

.....

4. Recalculate centroids:

After all data points are assigned to clusters, **recompute the centroids** of the clusters **by calculating the mean** of all the points in each cluster.

The **centroid** is **updated** to the **new value**.

5. Repeat assignment and update:

Repeat steps 3 and 4. **Reassign each data point** to the nearest centroid and **update the centroids** based on the new cluster memberships.

6. Stop when convergence is reached:

The **algorithm stops** when the **centroids no longer change**, meaning the clusters are stable.

At this point, we will have the **final clusters**.

Single Dimension K-Means Example

Imagine we have the following dataset consisting of **single-dimensional data points**:

Data points: [2, 3, 10, 12, 20]

We want to group these points into 2 clusters ($k=2$).

Step 1: Choose the number of clusters ($k=2$)

We choose to divide the dataset into **2 clusters**.

.....

Step 2: Initialize the centroids Randomly choose **two points** from the dataset as **initial centroids**.

Suppose we randomly select:

Centroid 1: 3
Centroid 2: 12

Step 3: Assign data points to the nearest centroid For each data point, calculate its distance from the two centroids and assign it to the nearest Centroid.

2:
Distance to **Centroid 1** (3): $|2 - 3| = 1$
Distance to **Centroid 2** (12): $|2 - 12| = 10$
Assign Data point 2 to Centroid 1.

....

3

Distance to **Centroid 1** (3): $|3 - 3| = 0$

Distance to **Centroid 2** (12): $|3 - 12| = 9$

Assign 3 to Centroid 1.

10

Distance to **Centroid 1** (3): $|10 - 3| = 7$

Distance to **Centroid 2** (12): $|10 - 12| = 2$

Assign 10 to Centroid 2

12

Distance to **Centroid 1** (3): $|12 - 3| = 9$

Distance to **Centroid 2** (12): $|12 - 12| = 0$

Assign 12 to Centroid 2.

.....

20

Distance to **Centroid 1** (3): $|20 - 3| = 17$

Distance to **Centroid 2** (12): $|20 - 12| = 8$

Assign 20 to Centroid 2.

Result:

Cluster 1: [2, 3]

Cluster 2: [10, 12, 20]

.....

Step 4: Recalculate centroids

Now, we recalculate the centroids by finding the mean of the points in each cluster

New Centroid 1

Mean of points in Cluster 1 (2, 3):

New Centroid 1 = $\frac{2+3}{2} = 2.5$

New Centroid 2

Mean of points in Cluster 2 (10, 12, 20):

New Centroid 2 = $\frac{10+12+20}{3} = 14$

.....

Step 5: Reassign points to the new centroids:

Recalculate the distance of each point to the new centroids (2.5 and 14):

2

Distance to Centroid 1 (2.5): $|2 - 2.5| = 0.5$

Distance to Centroid 2 (14): $|2 - 14| = 12$

Assign 2 to Centroid 1.

3

Distance to Centroid 1 (2.5): $|3 - 2.5| = 0.5$

Distance to Centroid 2 (14): $|3 - 14| = 11$

Assign 3 to Centroid 1.

.....

10

Distance to **Centroid 1 (2.5):** $|10 - 2.5| = 7.5$

Distance to **Centroid 2 (14):** $|10 - 14| = 4$

Assign 10 to Centroid 2.

12

Distance to **Centroid 1 (2.5):** $|12 - 2.5| = 9.5$

Distance to **Centroid 2 (14):** $|12 - 14| = 2$

Assign 12 to Centroid 2.

20

Distance to **Centroid 1 (2.5):** $|20 - 2.5| = 17.5$

Distance to **Centroid 2 (14):** $|20 - 14| = 6$

Assign 20 to Centroid 2.

.....

Result:

Cluster 1: [2, 3]

Cluster 2: [10, 12, 20]

Step 6: Recalculate centroids again by finding the mean of the points in each cluster

New Centroid 1

Mean of points in Cluster 1 (2, 3):

$$\text{New Centroid 1} = \frac{2+3}{2} = 2.5$$

New Centroid 2

Mean of points in Cluster 2 (10, 12, 20):

$$\text{New Centroid 2} = \frac{10+12+20}{3} = 14$$

.....

Centroid 1 remains at **2.5** (no change)

Centroid 2 remains at **14** (no change)

Since the **centroids did not change**, the algorithm has **converged**, and we have the **final clusters**.

Final Clusters:

Cluster 1: [2, 3]

Cluster 2: [10, 12, 20]

K-Means Problem-2 Dimensional

We now have six points:

1. $P_1 = (2, 10)$
2. $P_2 = (2, 5)$
3. $P_3 = (8, 4)$
4. $P_4 = (5, 8)$
5. $P_5 = (7, 5)$
6. $P_6 = (6, 4)$

Initial Centroids:

- $C_1 = (2, 10)$
- $C_2 = (5, 8)$

.....

Step 1: Compute Distance of Each Point to Centroids

Using the Euclidean distance formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

x1 y1

Point	$d(P, C_1 = (2, 10))$	$d(P, C_2 = (5, 8))$
P_1	$\sqrt{(2 - 2)^2 + (10 - 10)^2} = 0$	$\sqrt{(5 - 2)^2 + (8 - 10)^2} = \sqrt{9 + 4} = 3.6$
P_2	$\sqrt{(2 - 2)^2 + (5 - 10)^2} = 5$	$\sqrt{(5 - 2)^2 + (8 - 5)^2} = \sqrt{9 + 9} = 4.2$
P_3	$\sqrt{(8 - 2)^2 + (4 - 10)^2} = \sqrt{36 + 36} = 8.5$	$\sqrt{(5 - 8)^2 + (8 - 4)^2} = \sqrt{9 + 16} = 5$
P_4	$\sqrt{(5 - 2)^2 + (8 - 10)^2} = \sqrt{9 + 4} = 3.6$	$\sqrt{(5 - 5)^2 + (8 - 8)^2} = 0$
P_5	$\sqrt{(7 - 2)^2 + (5 - 10)^2} = \sqrt{25 + 25} = 7.1$	$\sqrt{(5 - 7)^2 + (8 - 5)^2} = \sqrt{4 + 9} = 3.6$
P_6	$\sqrt{(6 - 2)^2 + (4 - 10)^2} = \sqrt{16 + 36} = 7.2$	$\sqrt{(5 - 6)^2 + (8 - 4)^2} = \sqrt{1 + 16} = 4.1$

.....

Step 2: Assign Points to Clusters

Based on the minimum distance:

- Cluster 1 (C_1): P_1, P_2
- Cluster 2 (C_2): P_3, P_4, P_5, P_6

....

Step 3: Update Centroids

Compute the mean of the points in each cluster:

1. **Cluster 1:** Points $P_1 = (2, 10), P_2 = (2, 5)$

$$C'_1 = \left(\frac{2+2}{2}, \frac{10+5}{2} \right) = (2, 7.5)$$

2. **Cluster 2:** Points $P_3 = (8, 4), P_4 = (5, 8), P_5 = (7, 5), P_6 = (6, 4)$

$$C'_2 = \left(\frac{8+5+7+6}{4}, \frac{4+8+5+4}{4} \right) = \left(\frac{26}{4}, \frac{21}{4} \right) = (6.5, 5.25)$$

New Centroids:

- $C'_1 = (2, 7.5)$
- $C'_2 = (6.5, 5.25)$

.....

Step 1: Recompute Distances

Point	$d(P, C'_1 = (2, 7.5))$	$d(P, C'_2 = (6.5, 5.25))$
$P_1 = (2, 10)$	$\sqrt{(2-2)^2 + (10-7.5)^2} = \sqrt{0 + 6.25} = 2.5$	$\sqrt{(6.5-2)^2 + (5.25-10)^2} = \sqrt{20.25 + 22.75} = 6.9$
$P_2 = (2, 5)$	$\sqrt{(2-2)^2 + (5-7.5)^2} = \sqrt{0 + 6.25} = 2.5$	$\sqrt{(6.5-2)^2 + (5.25-5)^2} = \sqrt{20.25 + 0.0625} = 4.5$
$P_3 = (8, 4)$	$\sqrt{(8-2)^2 + (4-7.5)^2} = \sqrt{36 + 12.25} = 6.7$	$\sqrt{(6.5-8)^2 + (5.25-4)^2} = \sqrt{2.25 + 1.56} = 2.0$
$P_4 = (5, 8)$	$\sqrt{(5-2)^2 + (8-7.5)^2} = \sqrt{9 + 0.25} = 3.0$	$\sqrt{(6.5-5)^2 + (5.25-8)^2} = \sqrt{2.25 + 7.56} = 3.1$
$P_5 = (7, 5)$	$\sqrt{(7-2)^2 + (5-7.5)^2} = \sqrt{25 + 6.25} = 5.6$	$\sqrt{(6.5-7)^2 + (5.25-5)^2} = \sqrt{0.25 + 0.0625} = 0.56$
$P_6 = (6, 4)$	$\sqrt{(6-2)^2 + (4-7.5)^2} = \sqrt{16 + 12.25} = 5.1$	$\sqrt{(6.5-6)^2 + (5.25-4)^2} = \sqrt{0.25 + 1.56} = 1.3$

.....

Step 2: Reassign Points to Clusters

Based on the minimum distances:

- Cluster 1 ($C'_1 = (2, 7.5)$): P_1, P_2, P_4
- Cluster 2 ($C'_2 = (6.5, 5.25)$): P_3, P_5, P_6

Step 3: Update Centroids

1. Cluster 1 ($P_1 = (2, 10), P_2 = (2, 5), P_4 = (5, 8)$):

$$C''_1 = \left(\frac{2 + 2 + 5}{3}, \frac{10 + 5 + 8}{3} \right) = \left(\frac{9}{3}, \frac{23}{3} \right) = (3, 7.67)$$

2. Cluster 2 ($P_3 = (8, 4), P_5 = (7, 5), P_6 = (6, 4)$):

$$C''_2 = \left(\frac{8 + 7 + 6}{3}, \frac{4 + 5 + 4}{3} \right) = \left(\frac{21}{3}, \frac{13}{3} \right) = (7, 4.33)$$

.....

Step 4: Check for Convergence

If centroids **remain the same**, we say the **algorithm has converged**.

- New centroids:
 - $C_1'' = (3, 7.67)$
 - $C_2'' = (7, 4.33)$

The **centroids** have **changed**, so **we would repeat the process**. If, in the **next iteration**, the centroids **no longer change**, the **algorithm will stop**.

The K-Means algorithm **iteratively updates** centroids and **reassigns points** to clusters.

After convergence:

- **Final Cluster 1:** Contains points close to C1
- **Final Cluster 2:** Contains points close to C2

Silhouette Clustering

Silhouette clustering is a method to evaluate the **quality of clustering** by measuring how **similar each data point** is to its own cluster compared to other clusters.

Silhouette Coefficient Formula

For a single data point i , the silhouette coefficient $S(i)$ is calculated as:

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Where:

- $a(i)$: The average distance between point i and all other points in the **same cluster** (intra-cluster distance).
- $b(i)$: The average distance between point i and all points in the **nearest neighboring cluster** (inter-cluster distance).

S(i) ranges from -1 to 1:

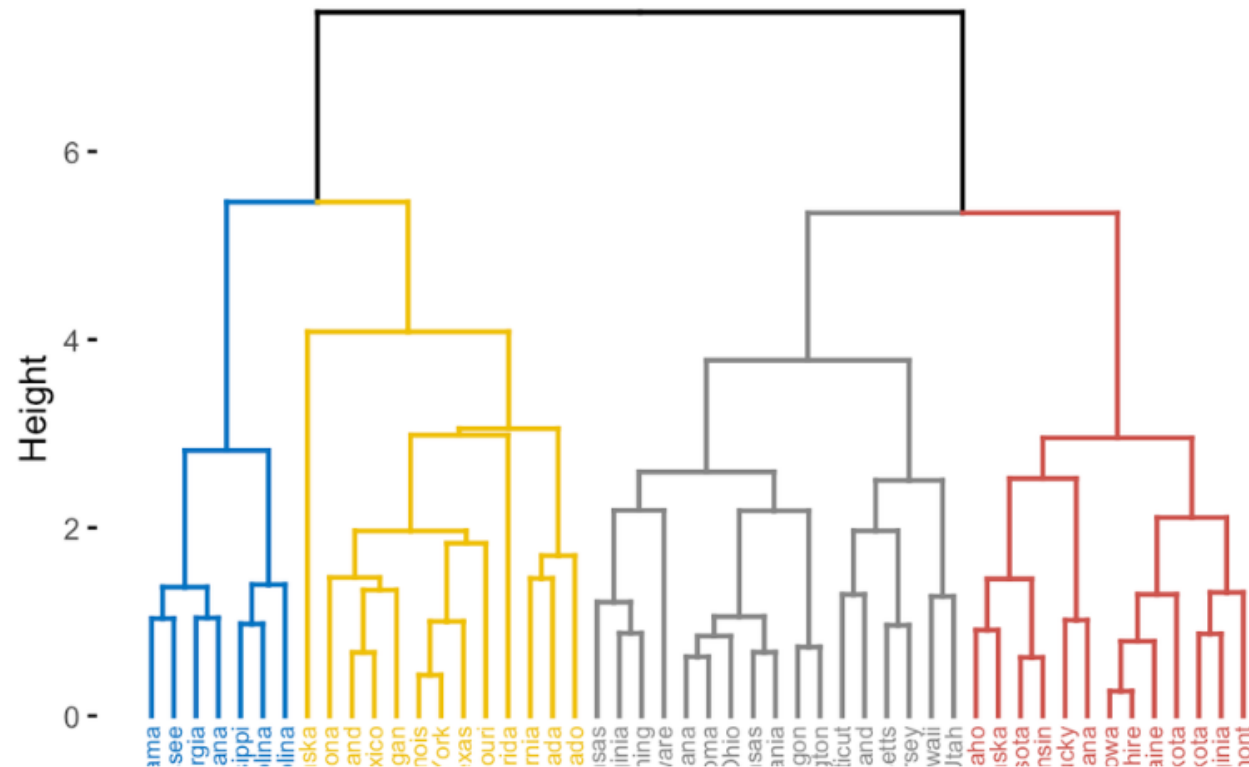
1: Perfectly clustered (point is well-matched to its own cluster and far from other clusters).

0: Overlapping clusters (point lies on or near the boundary between clusters).

Negative values: Poor clustering (point is closer to another cluster than its own).

Hierarchical Clustering

Hierarchical clustering is a method of clustering that builds a **hierarchy of clusters**, often **visualized** as a **tree-like structure** called a **dendrogram**.



.....

Types of Hierarchical clustering.

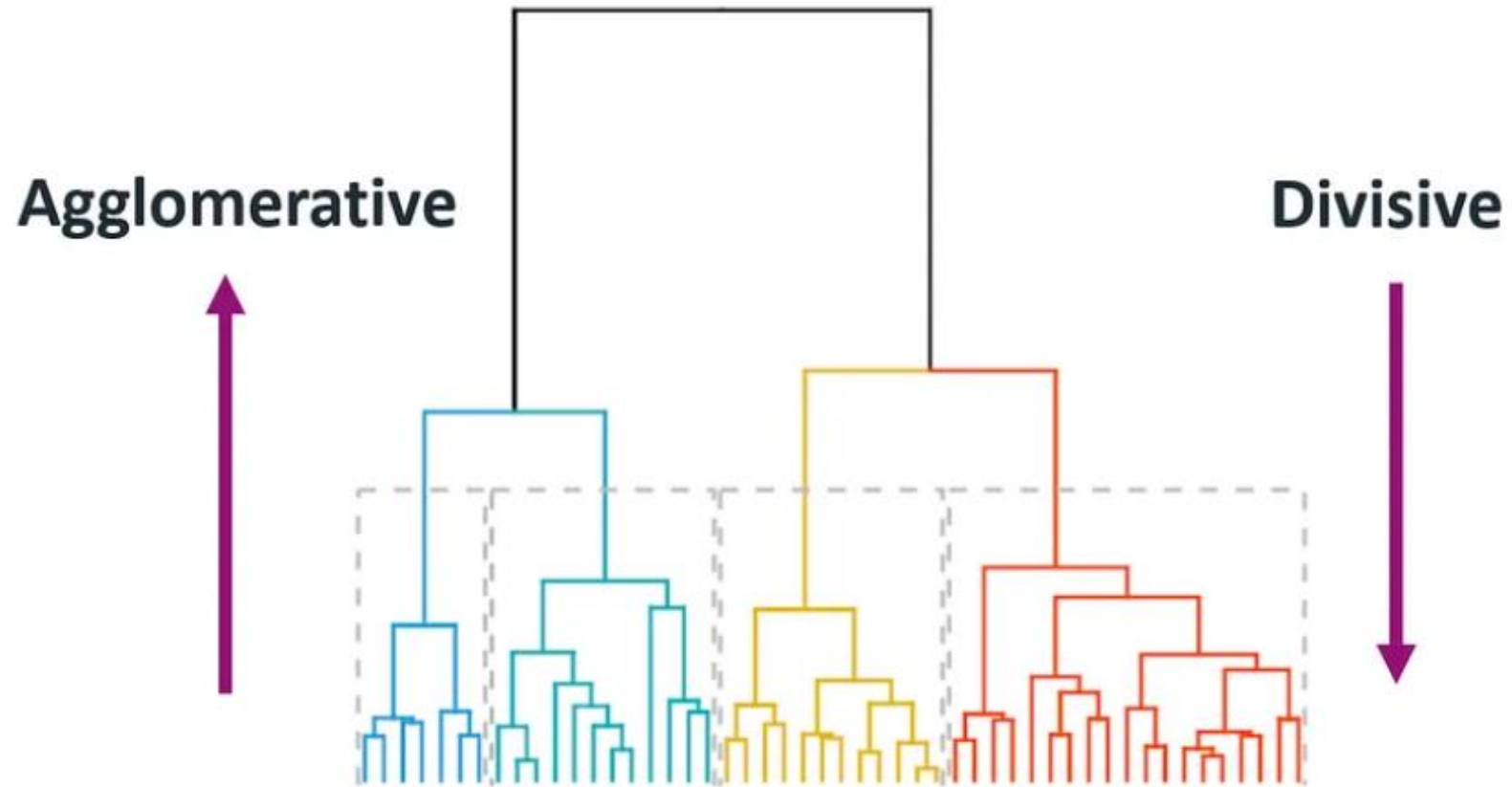
1. Agglomerative (Bottom-Up):

Each **data point** acts as a **cluster** initially, and then it **groups** the clusters one by one based on similarity.

2. Divisive (Top-Down):

Starts with all data points into a single cluster. and **splits them** into smaller clusters.

.....



Both this algorithm are exactly reverse of each other

Key parameters of Hierarchical Clustering

Distance Metric: Determines how the similarity between two points is measured (e.g., Euclidean, Manhattan).

Linkage Criteria: Determines how the distance between clusters is calculated.

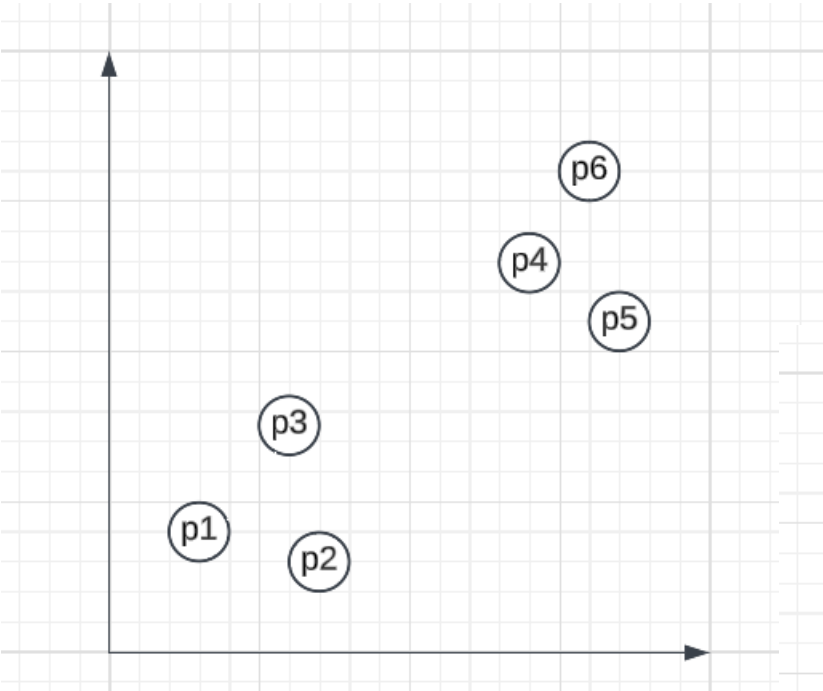
Single Linkage: Minimum distance between points in the clusters.

Complete Linkage: Maximum distance between points in the clusters.

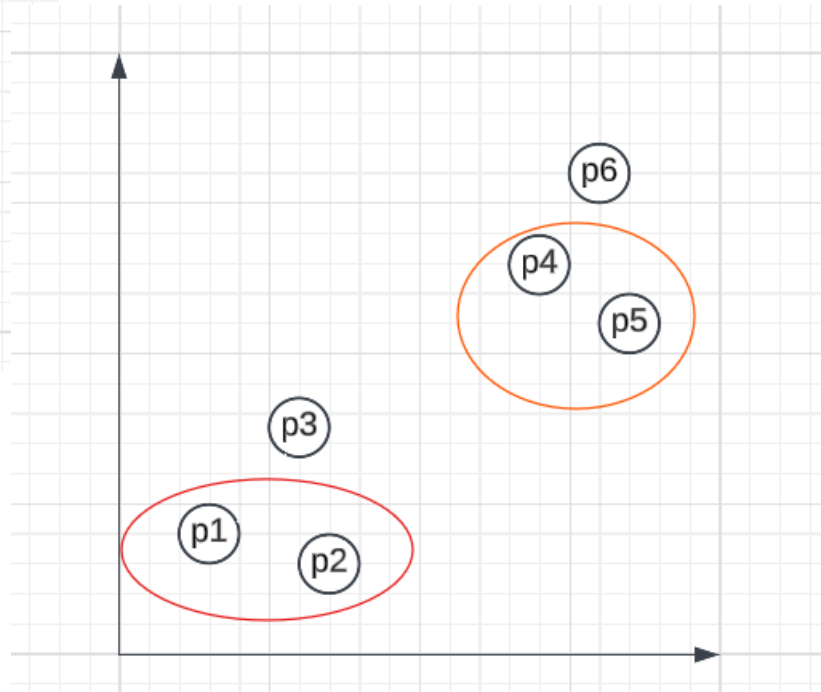
Average Linkage: Average distance between points in the clusters.

Agglomerative (Bottom-Up):

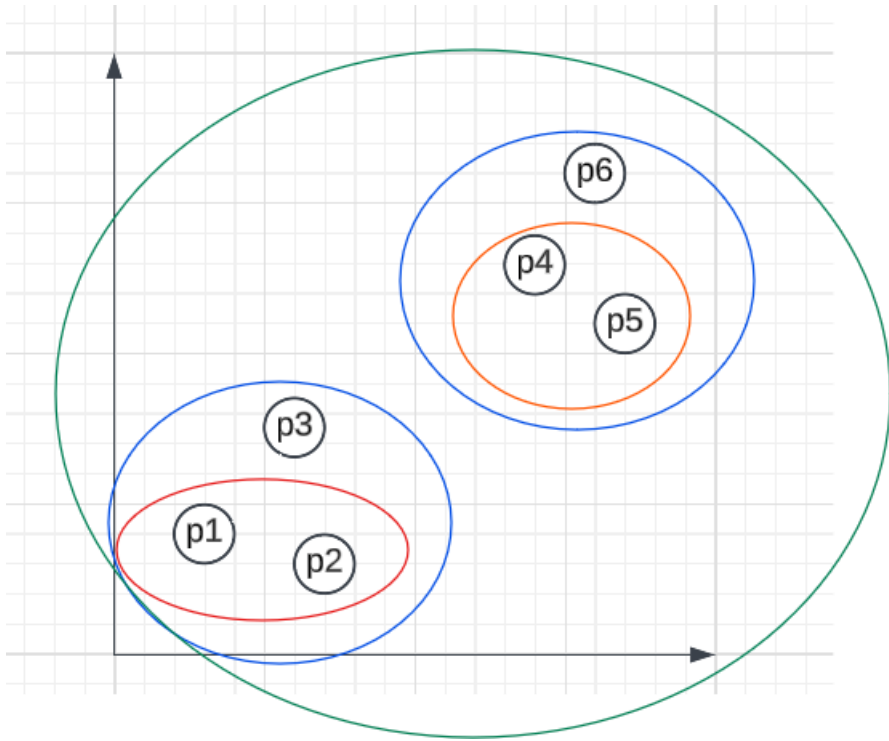
1. For each point initially we will consider it as a separate cluster.



2. Find the nearest point and combine to create a new cluster.

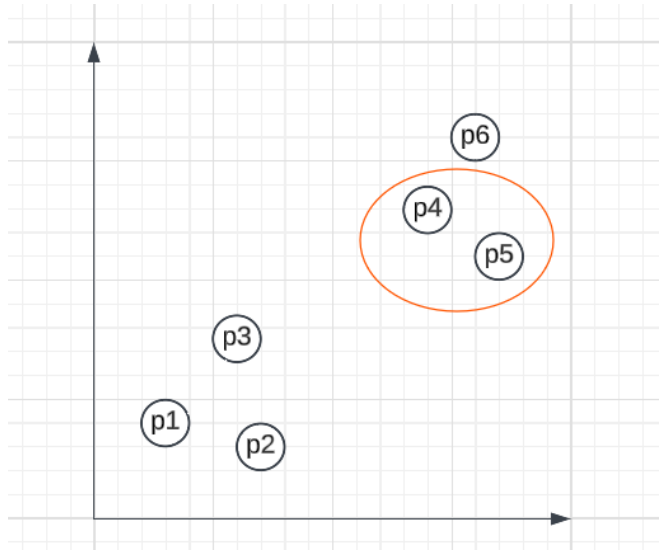


.....

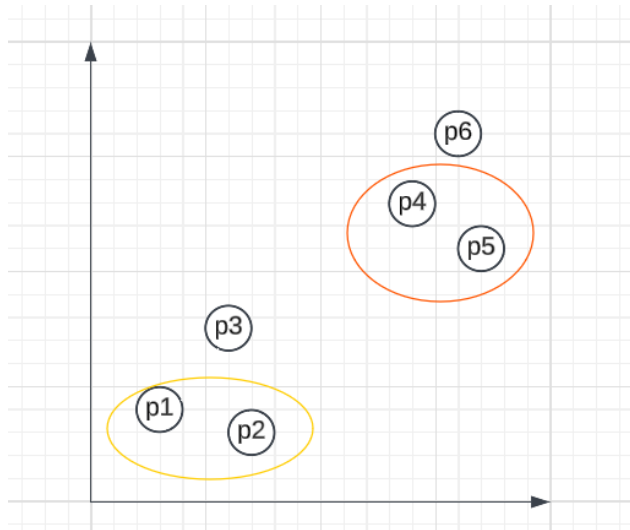
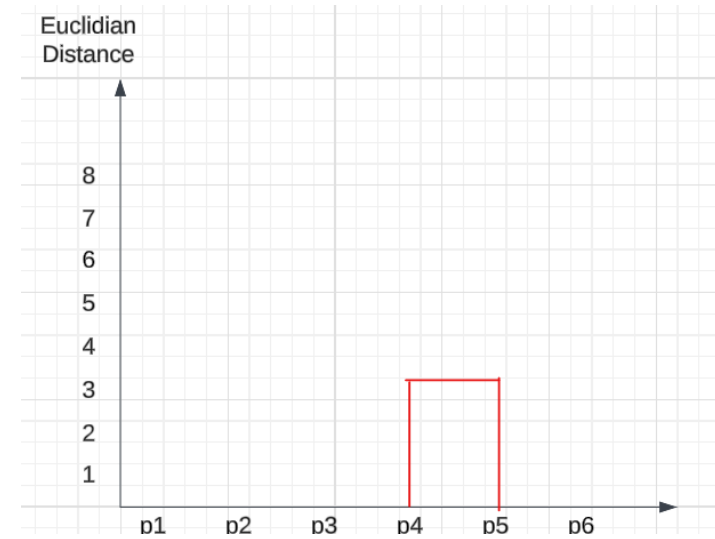


3. Keep doing the process until we combine to a single cluster.

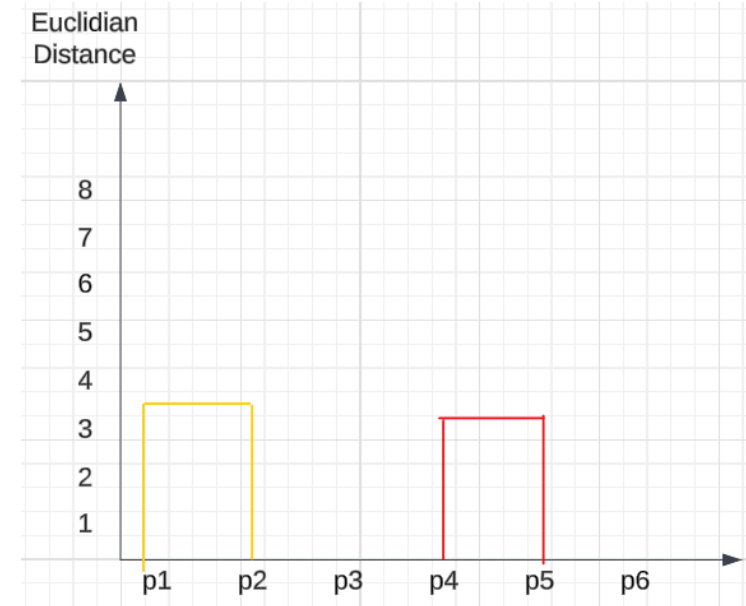
Dendrogram



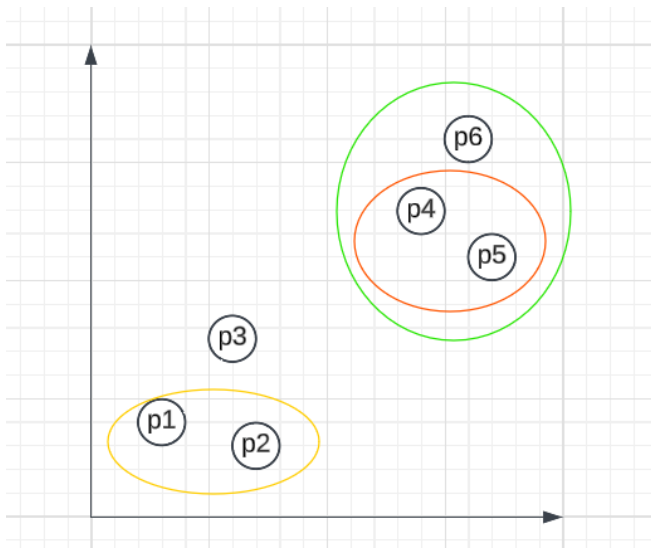
p4 and p5 are combined together



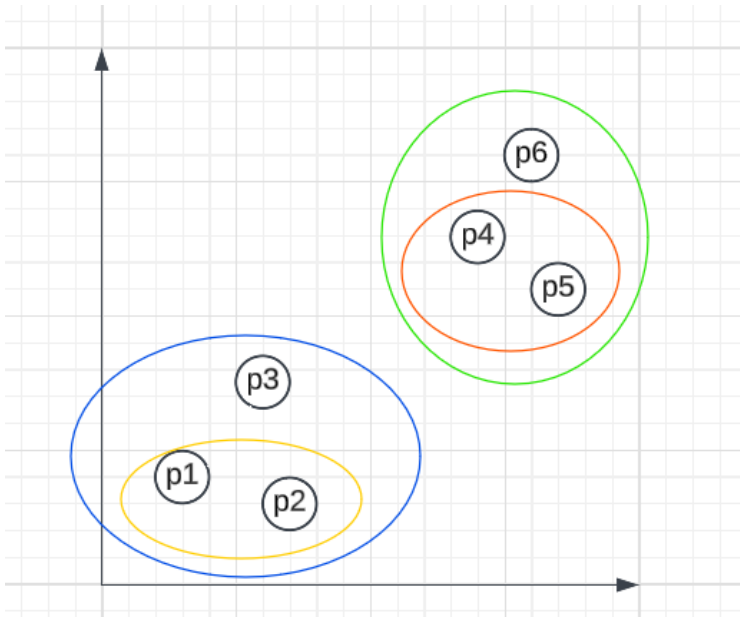
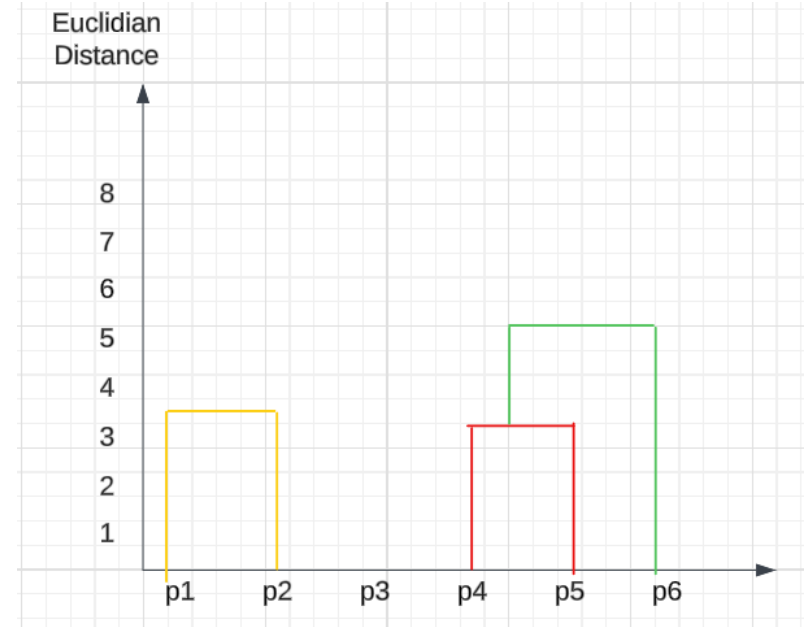
p1 and p2 are combined together



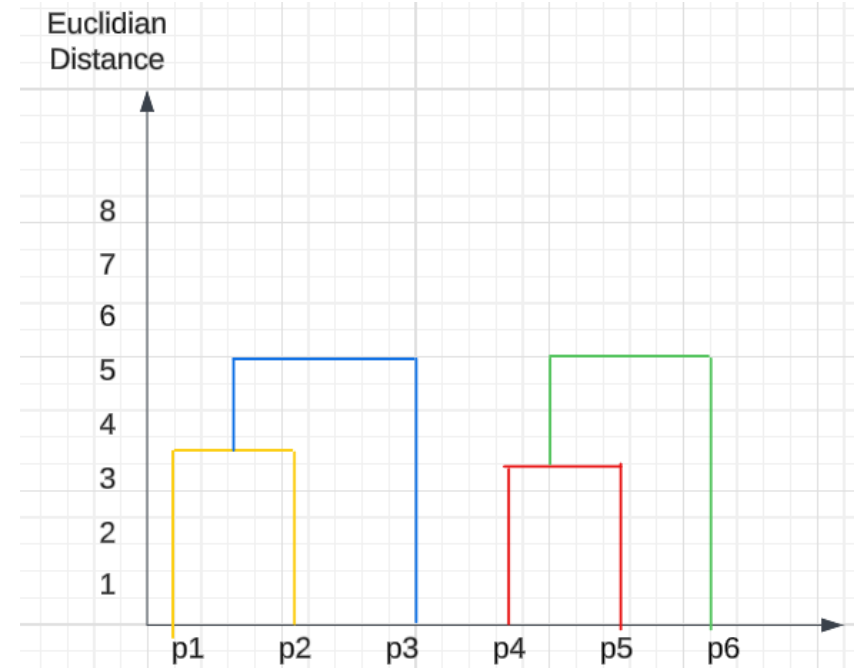
.....



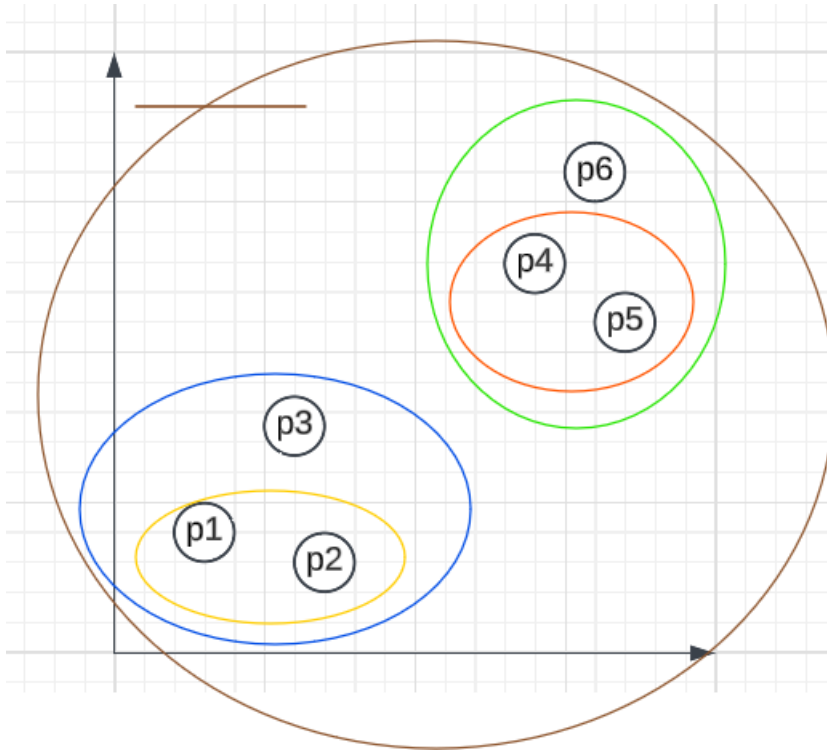
P4 ,p5 and p6 are combined together



P1 ,p2 and p3 are combined together

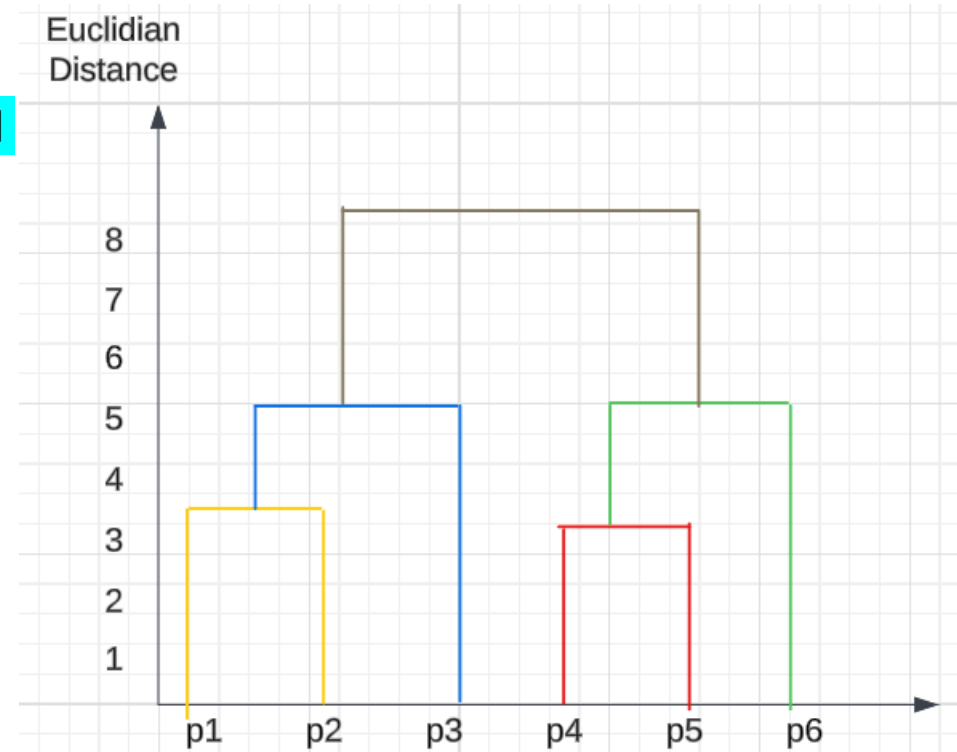


.....

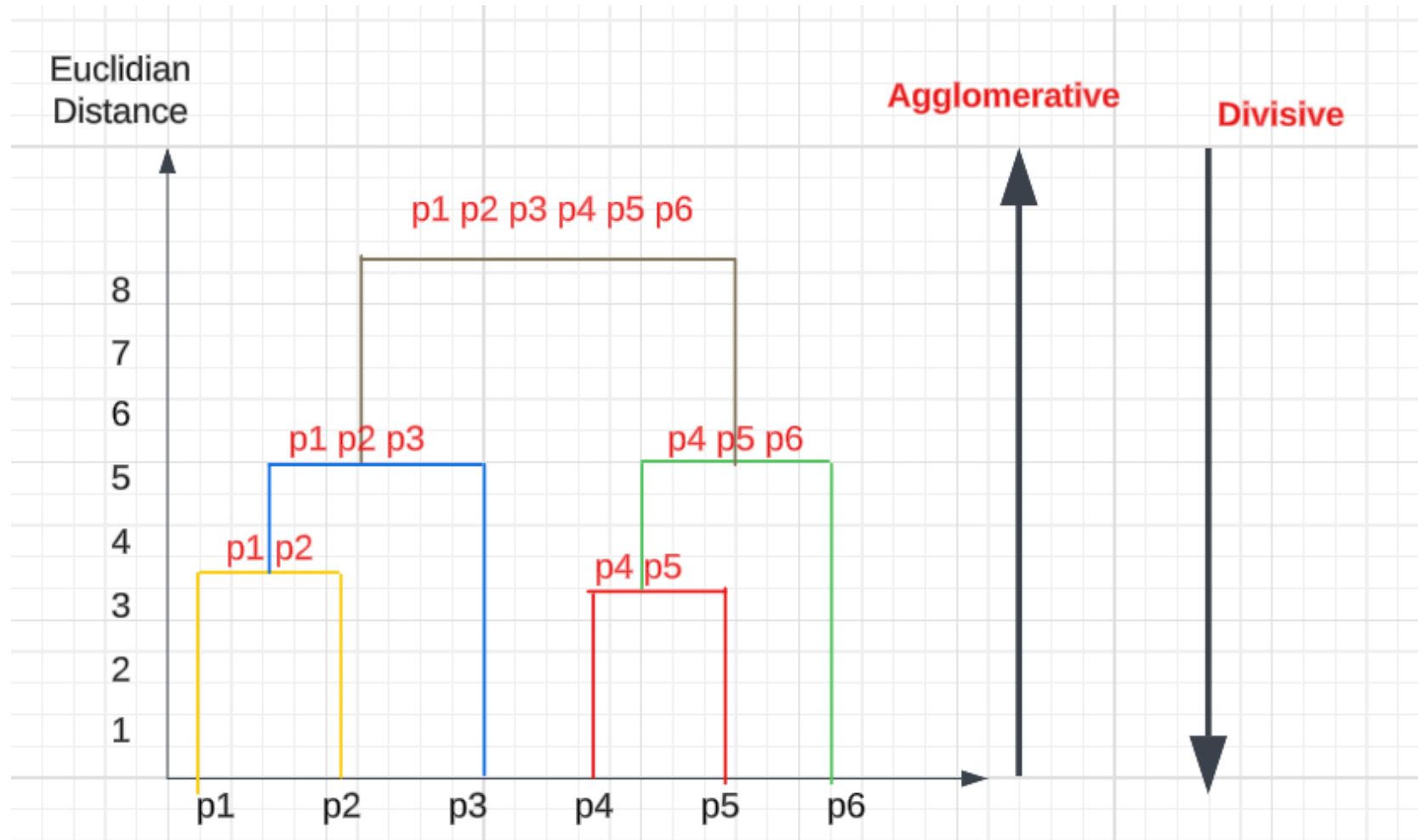


p1, p2, p3, p4, p5 and p6 are combined together to form a single cluster.

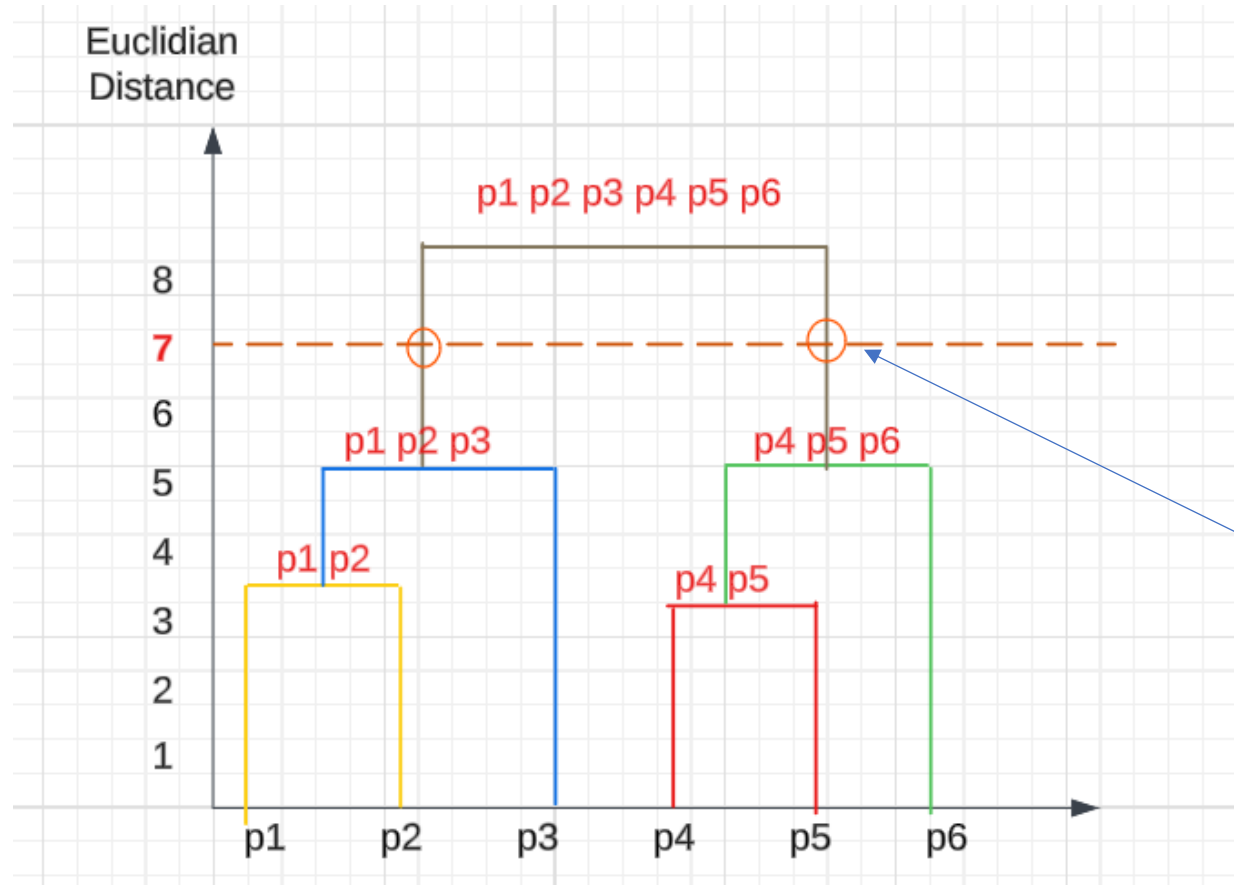
This is called **Agglomerative**



• • • • •



How we will determine the K value?



Let us set a **threshold value of 7.**

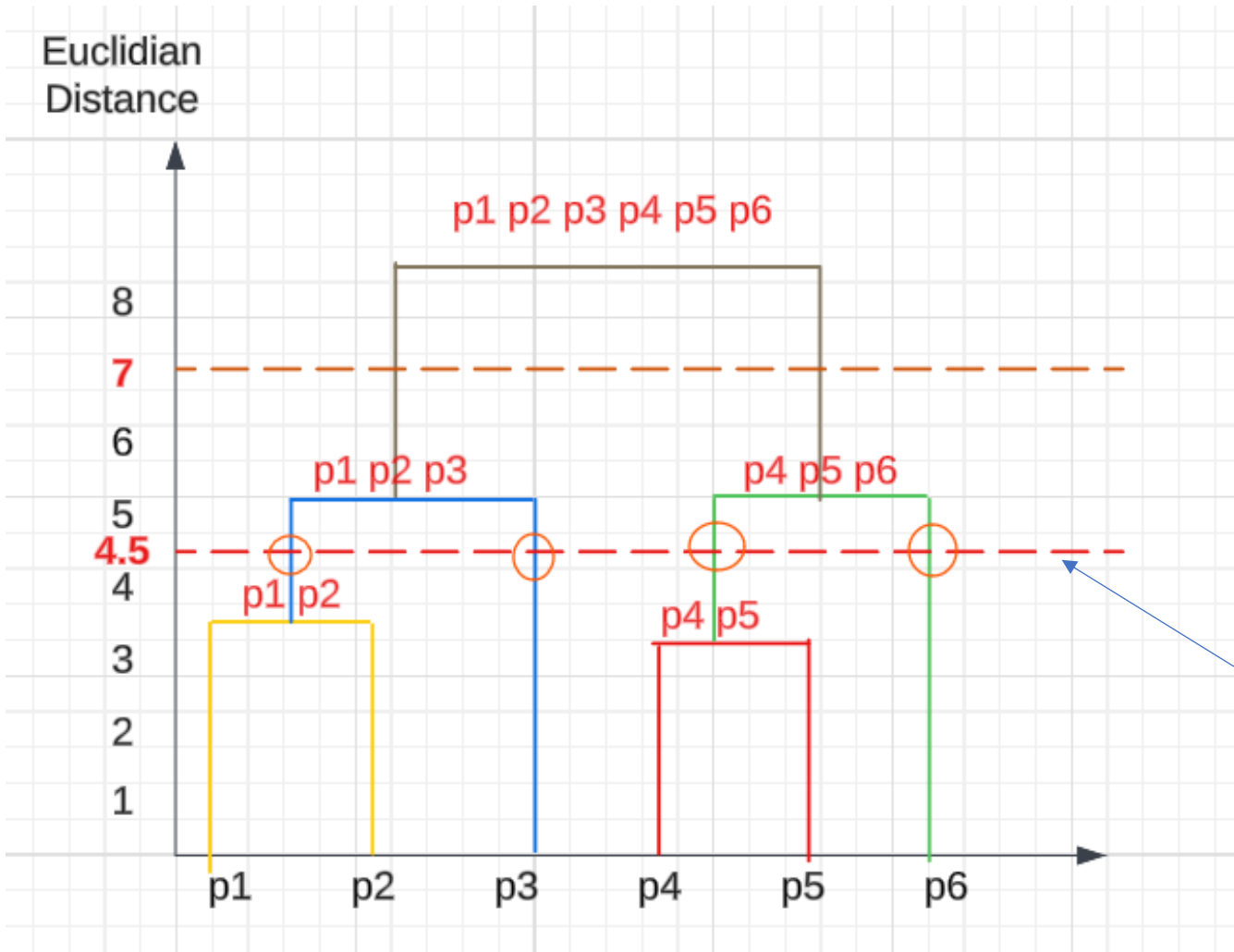
Next, draw a **horizontal line** at this threshold level.

The horizontal line **intersects two vertical lines in the dendrogram.**

Therefore, the value of **k (number of clusters)** is 2.

Threshold value is the **Euclidian distance in the Y axis**

.....



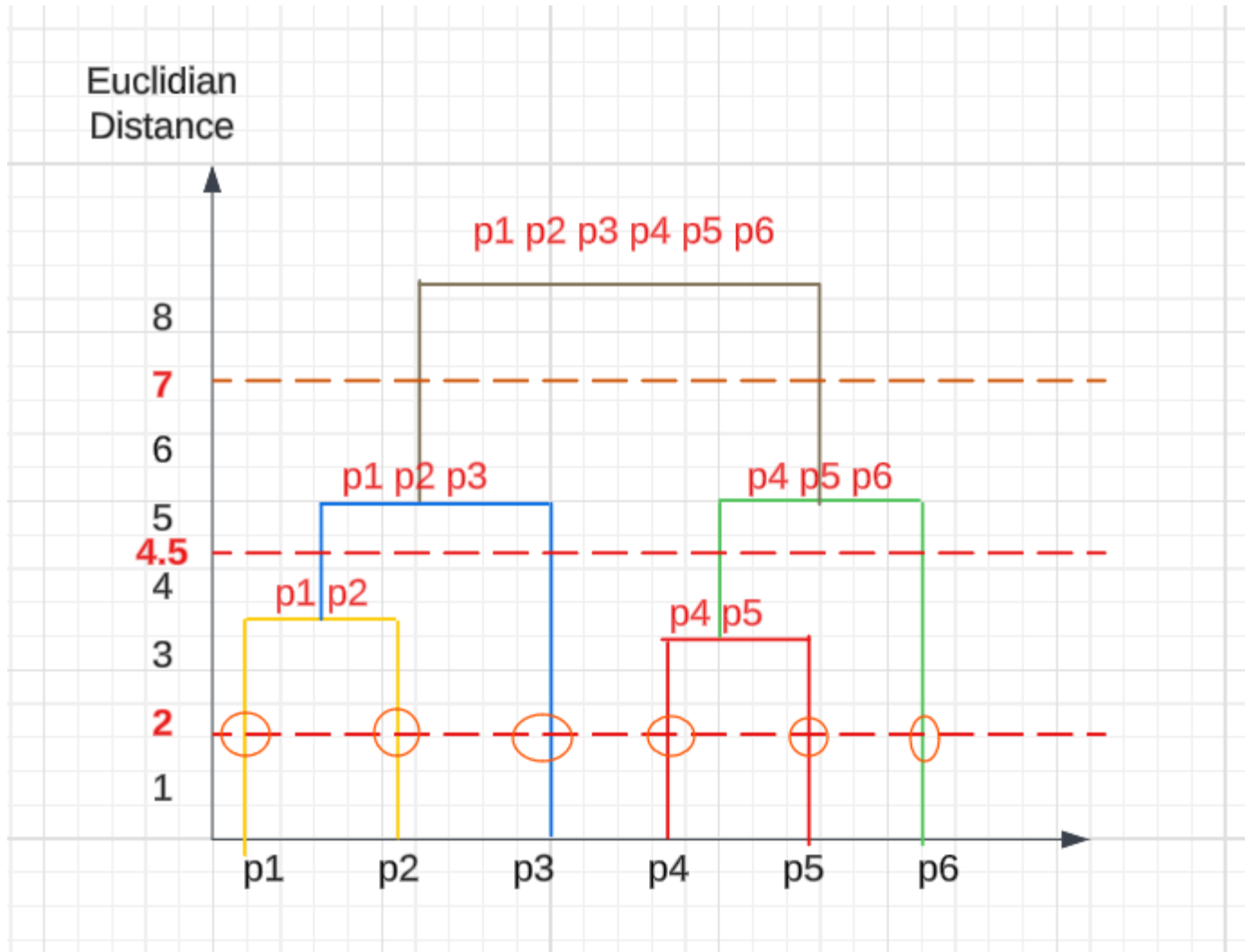
Let us set a **threshold value of 4.5**.

Next, draw a **horizontal line** at this threshold level.

The horizontal line **intersects four vertical lines in the dendrogram**.

Therefore, the value of **k (number of clusters)** is 4.

.....



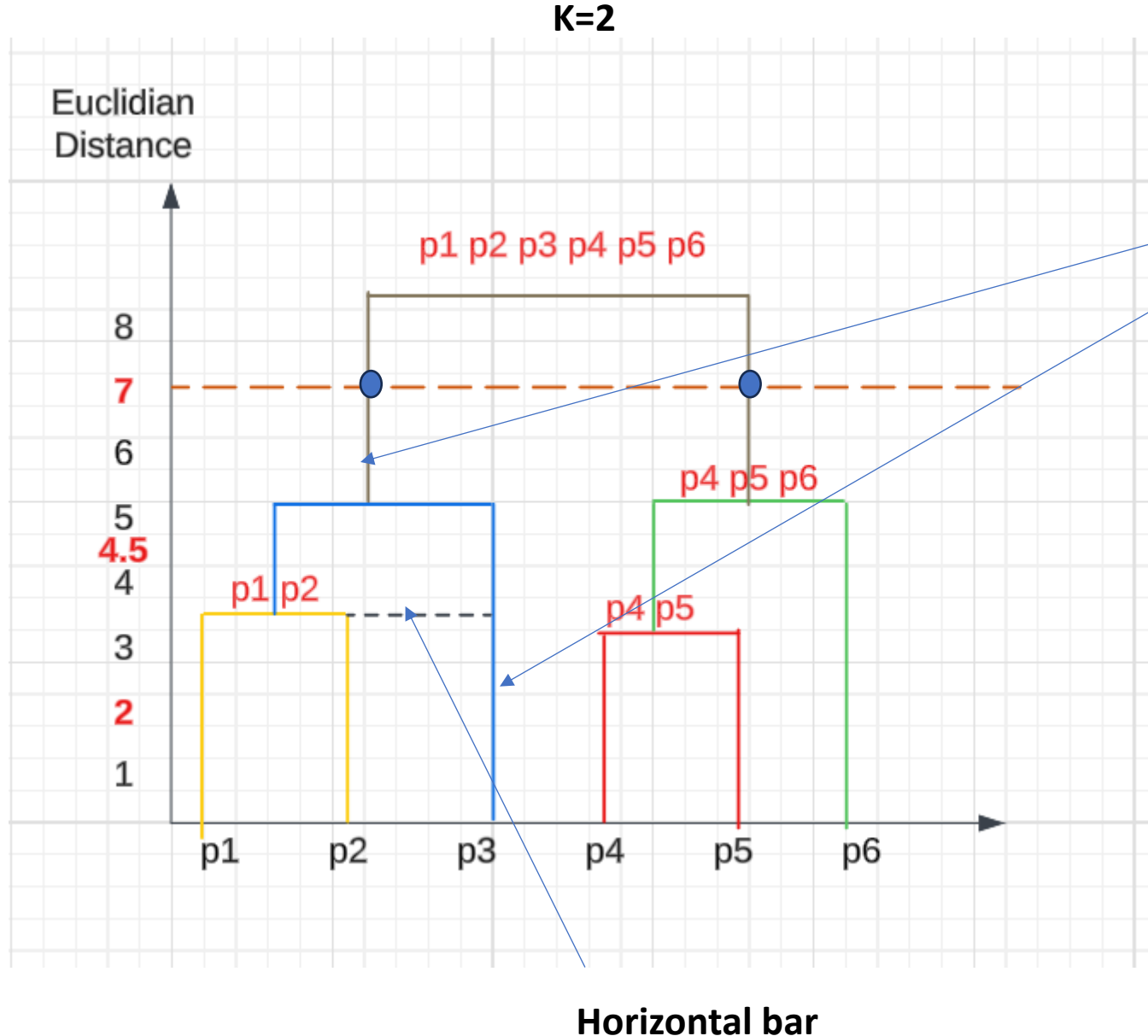
Let us set a **threshold value of 2**.

Next, draw a **horizontal line** at this threshold level.

The horizontal line **intersects six vertical lines in the dendrogram**.

Therefore, the value of **k (number of clusters)** is 6.

Selecting the Threshold is another important challenge



This 2 are the longest vertical line.
But the problem in the **blue colour** line is a **horizontal bar** will pass through it.

But it is not the problem with **brown colour** line.

So the longest line in the dendrogram is **brown colour line**. Now you **draw a horizontal line and fix the threshold value.**

Since this horizontal line passing through two values $k = 2$

CURE (Clustering Using Representatives)

Clustering

- Clustering groups data based on similarities
 - Similarity is defined using a distance measure (Euclidean, Jaccard, Cosine, etc)



CURE Algorithm

Rather than finding **Centroids** we need to find representative points, to **create clusters**

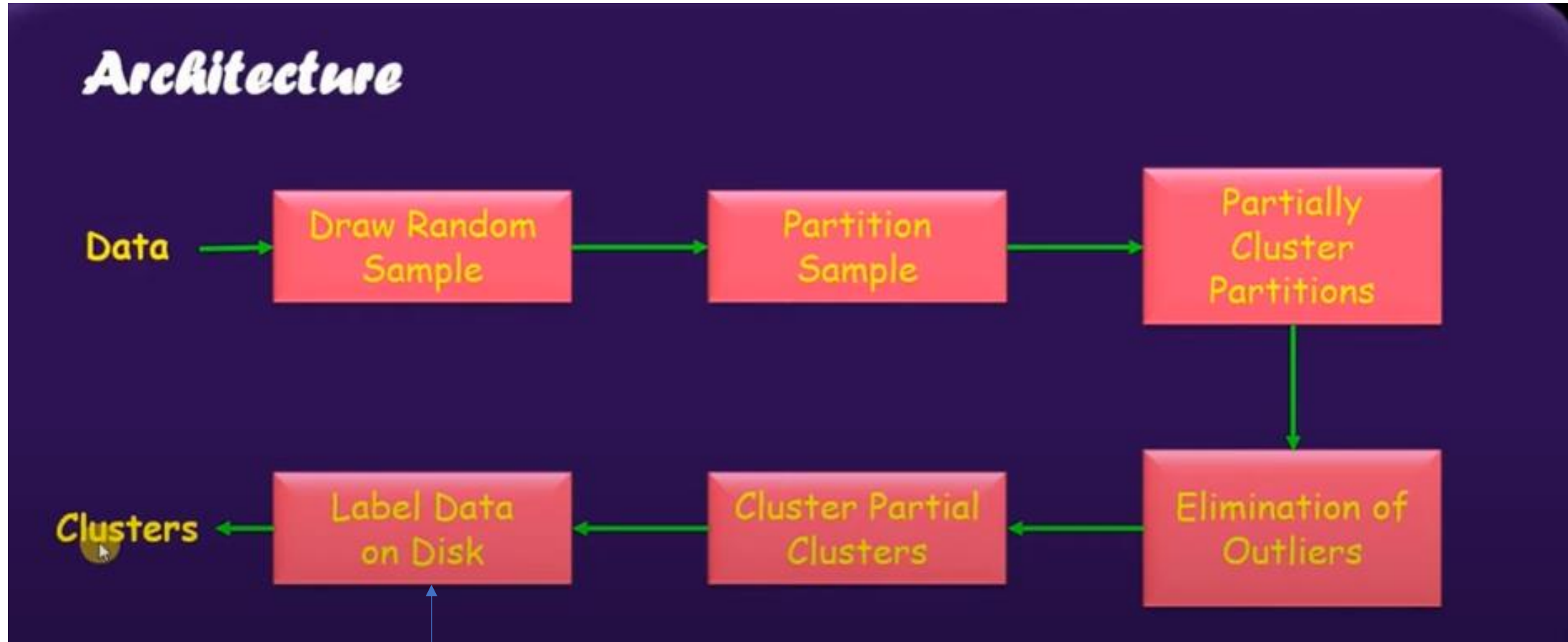
Hierarchical clustering and density-based methods like **DBSCAN** consider the relationships between **all points or boundary points**, rather than **relying solely on centroids**.

Overview

- CURE stands for **C**lustering **U**sing **R**epresentatives
- Specially designed to work efficiently on larger datasets
- Uses a collection of representative points to represent clusters
- Adopts a middle ground between centroid based and all-point extremes
- Capable of detecting clusters of any shape
- Detect outliers and remove it

like **K-means**, where clusters are formed around a **central point (centroid)**

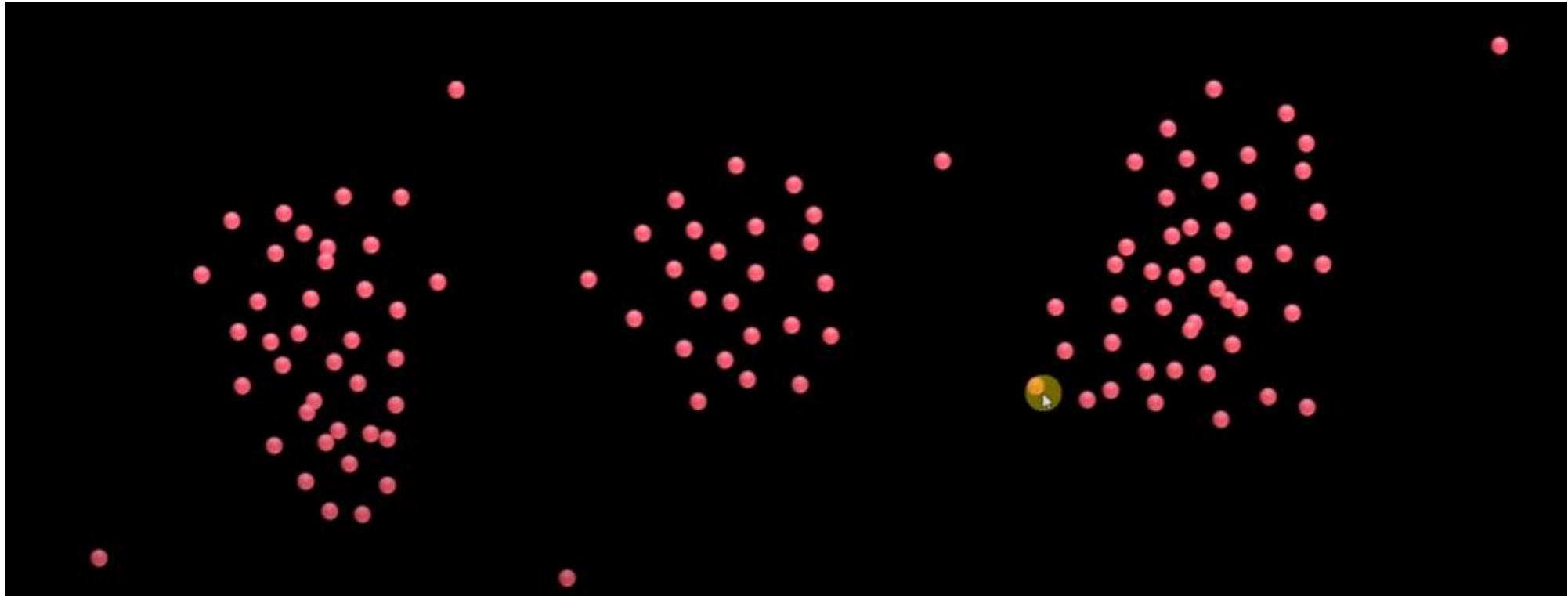
CURE Algorithm Architecture



Names that Represent the particular Cluster

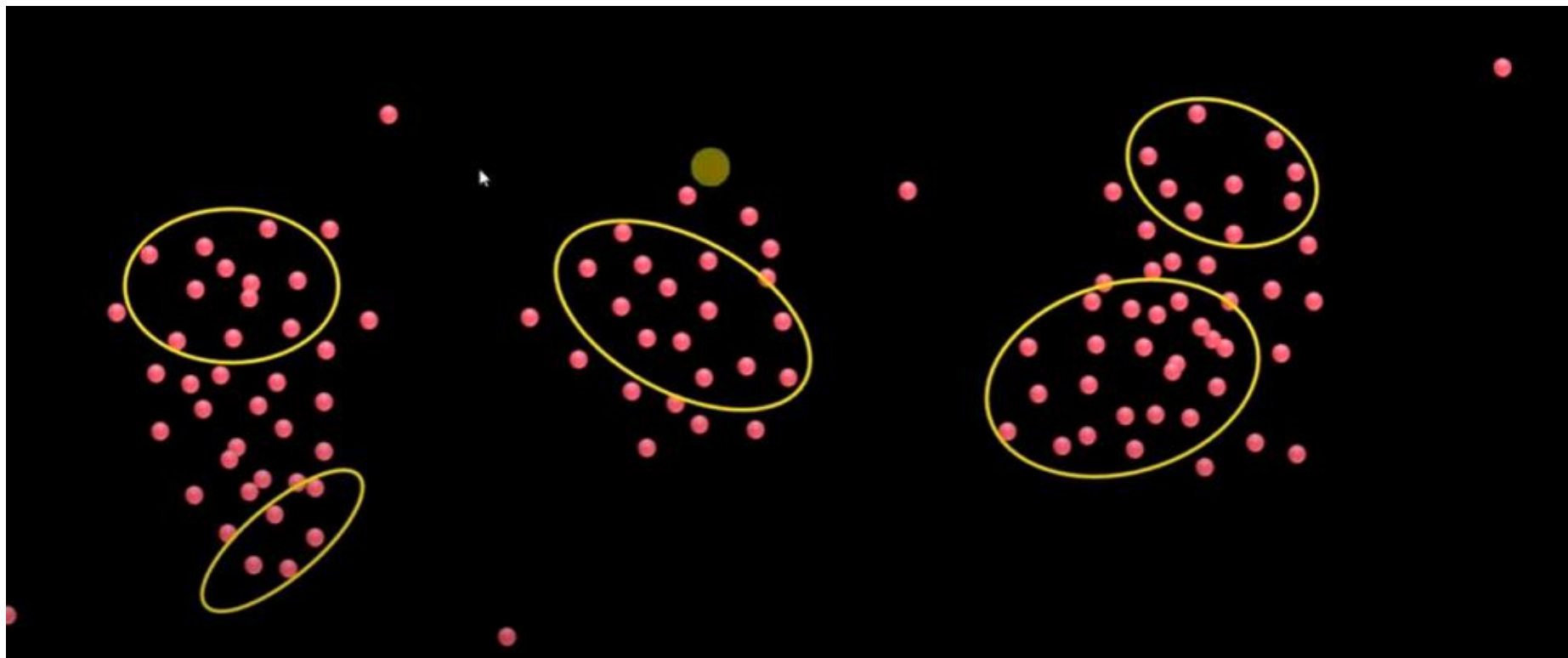
CURE Algorithm Example

Sample Data set



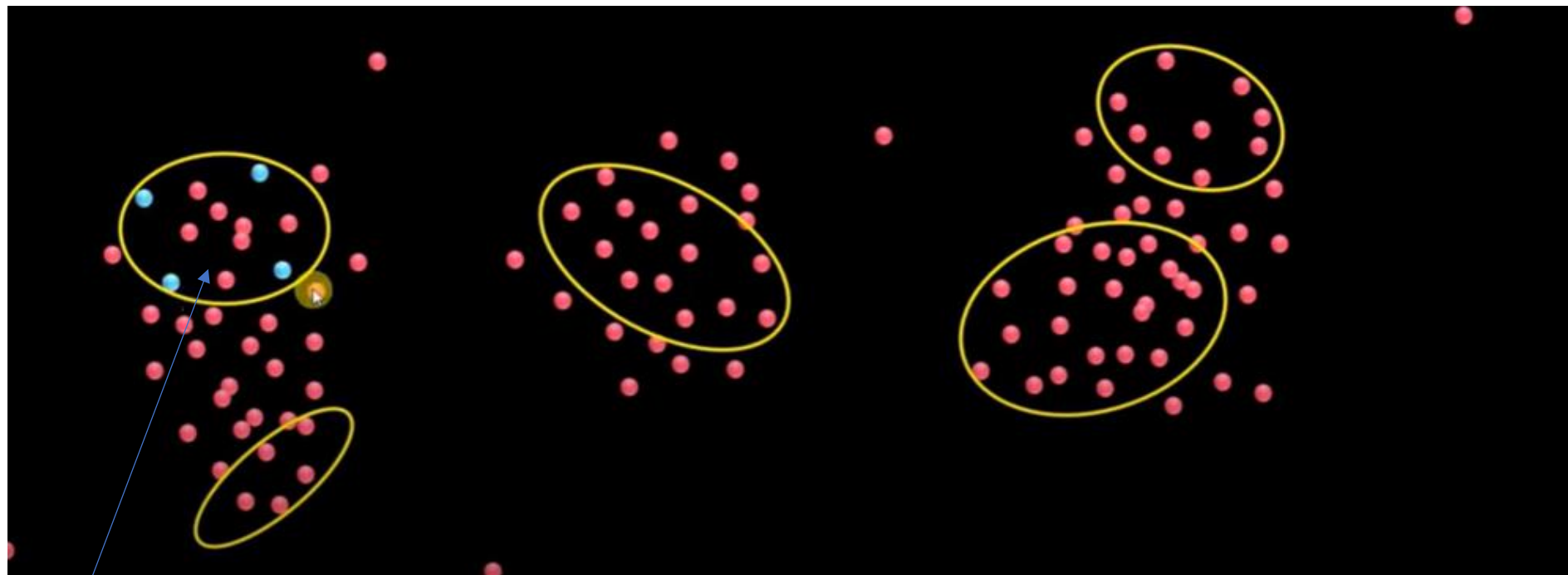
.....

1. Pick **Random Samples** and **Cluster** them



.....

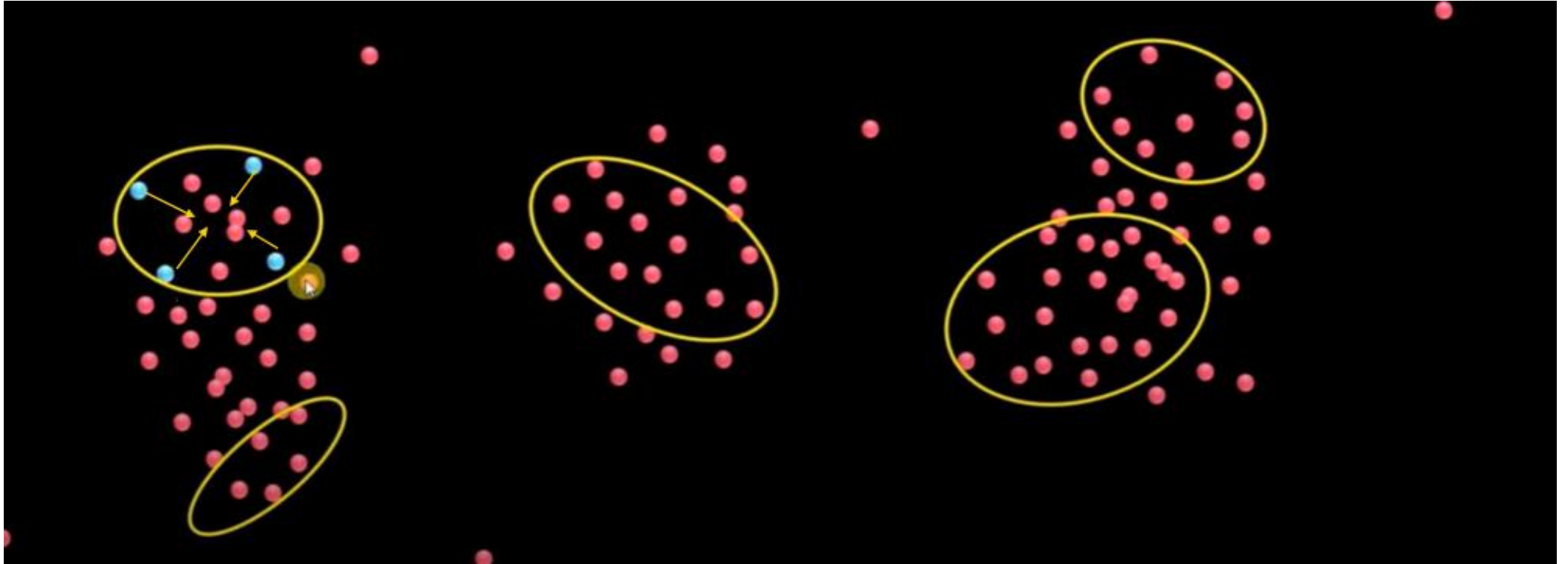
2. Choose **C Scatter points** in Each cluster . Let's take $C = 4$



Scatter points should be
far away from each other

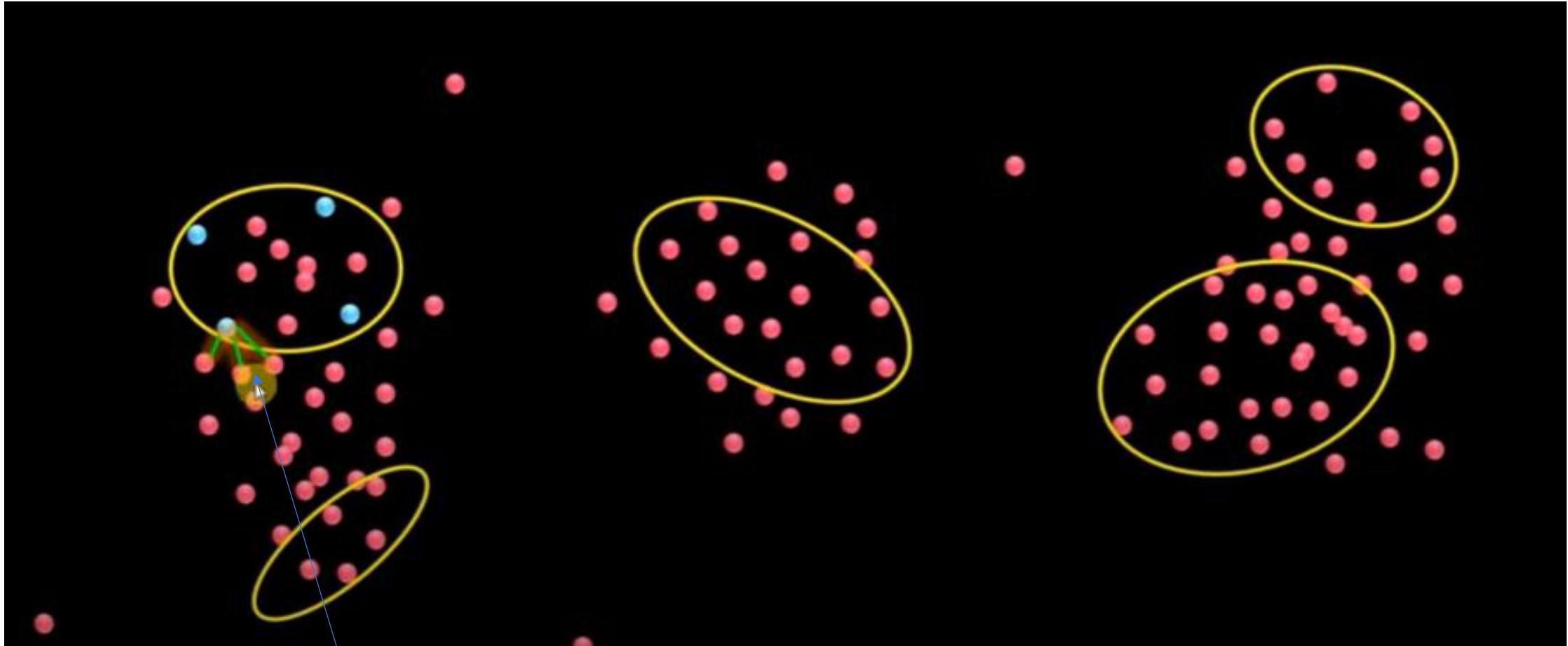
.....

3. These Scatter points should be shrunk towards the centroid in a **fraction of α where $0 < \alpha < 1$**
We will get a ***Dmin*** Distance



Dmin it is the distance between the **centroid position** and each **representative data** point position after shrinking it by **fraction of alpha**

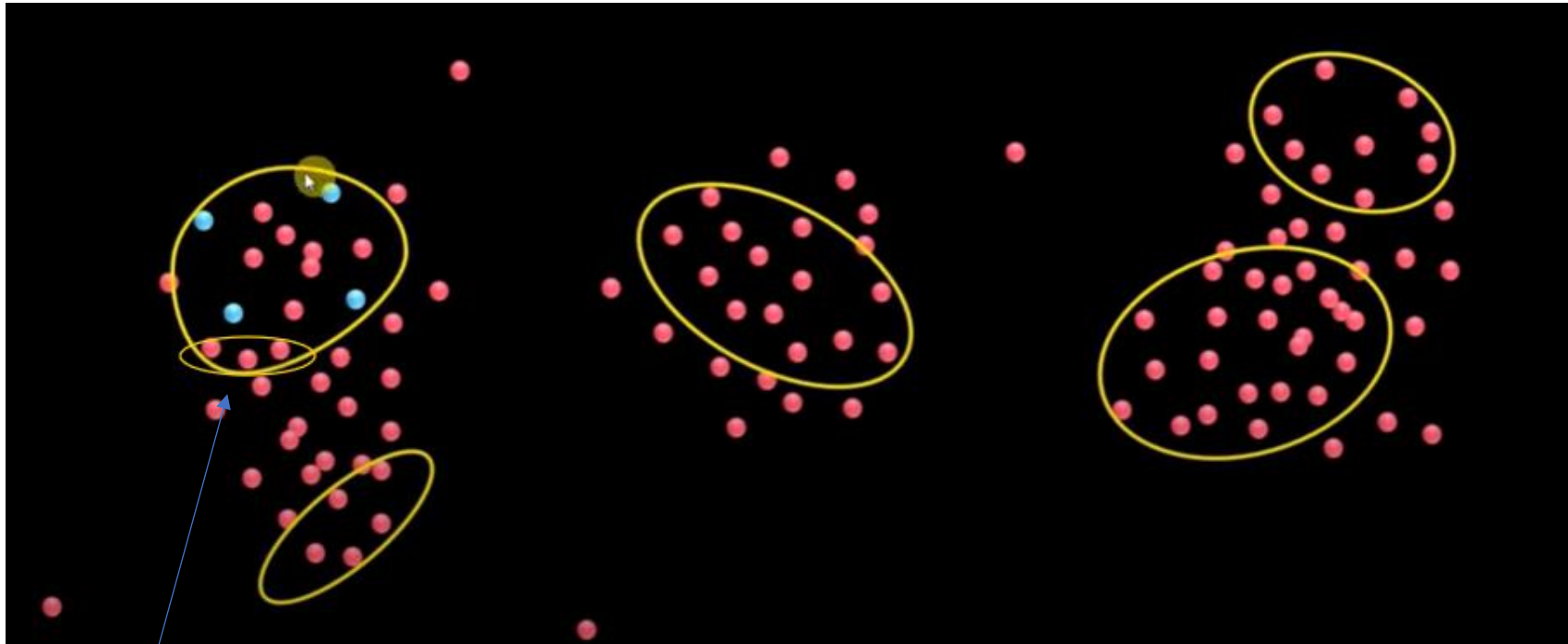
4. We have to use this **Dmin** distance for **cluster merging** approach by considering this data point as **representatives of the cluster**



Use **Dmin** distance as the condition for Merging

.....

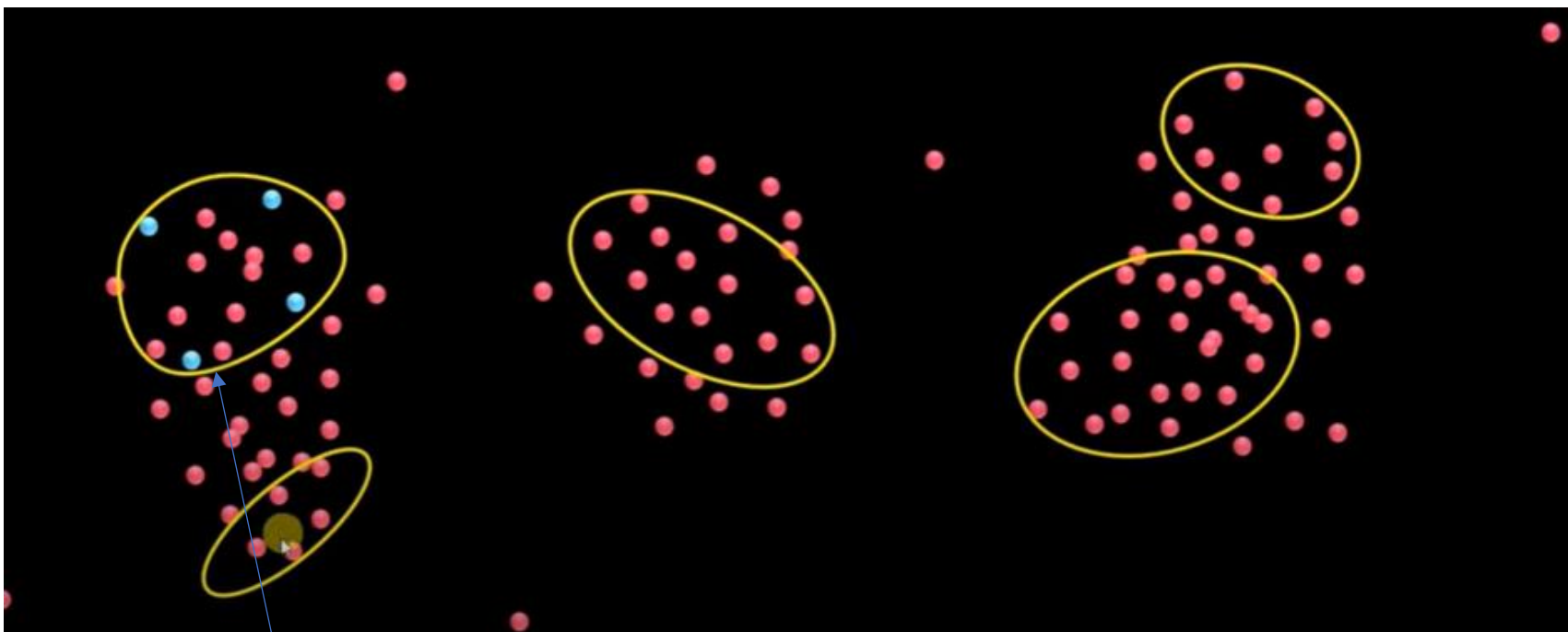
5. After New Merge ,**New points merged** are considered as **representative of New** cluster.



Since the distance is $\leq$ **Dmin**
distance we **clubbed** them in the
cluster

.....

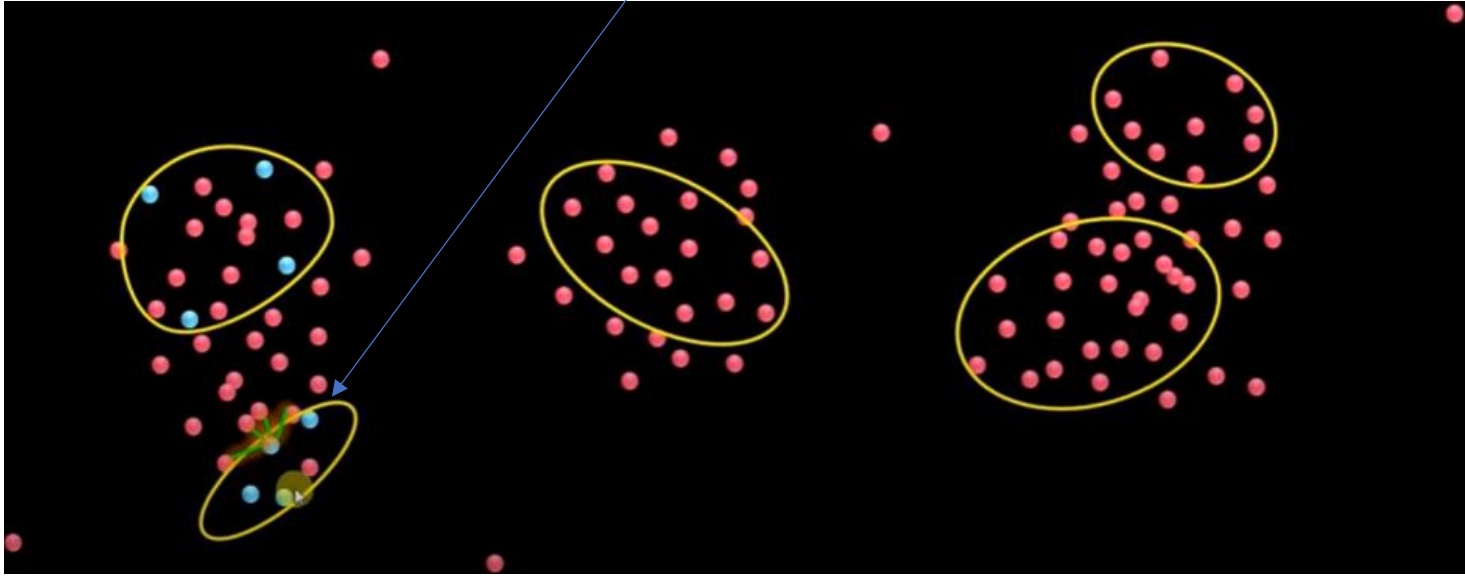
6. Finally, **cluster merging** will **stop** when **target K**(Number of clusters) is achieved.



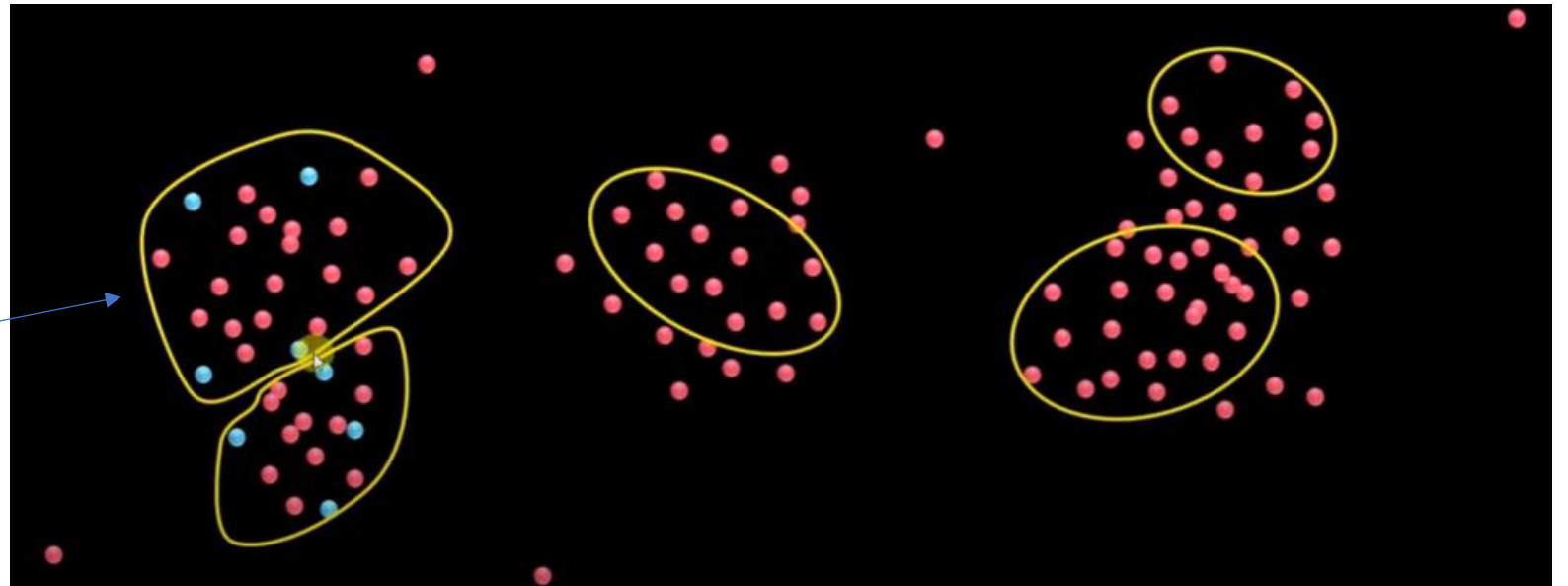
One of the **new data point merged** is considered as **new representative**

We have to follow the **previous steps** to other clusters too

.....



We can see the data points clustered in these two cluster



Kmeans vs Hierarchical clustering

Data size

Huge – K –Means Clustering

Small – Hierarchical clustering

Data type

Numerical data – K- Means

Variety data – Hierarchical clustering

Distance Metric:

Uses **Euclidean distance** to assign points to the nearest **centroid**, - **K-means**

Uses different **linkage criteria** to compute **distances between clusters**

Single linkage

Complete linkage

Average linkage

Centroid linkage